

Trường Đại học Khoa học tự nhiên - ĐHQG HCM



fit@hcmus

Khoa Công nghệ thông tin

BÁO CÁO ĐỒ ÁN

Mạng máy tính

PHÂN TÍCH GÓI TIN VỚI WIRESHARK

Ngày 29 tháng 12 năm 2024

Lớp 23CTT3

NHÓM 1A

23120233 - Nguyễn Lê Hữu Điền

23120262 - Tống Dương Thái Hoà

23120264 - Nguyễn Phúc Hoàng

Giảng viên hướng dẫn

Ths. Huỳnh Thụy Bảo Trân

Ths. Chung Thùy Linh

Mục lục

1	Giới thiệu	3
1.1	Thông tin thành viên	4
1.2	Phân chia công việc	4
2	HTTP	5
2.1	Nhìn vào những gói đầu tiên, máy client đang truy vấn IP của một tên miền. Cho biết tên miền của website là gì? Cho biết hai địa chỉ IPs của website này?	5
2.2	Có bao nhiêu gói tin (packets) mà Server đã gửi cho máy client tính từ lúc bắt đầu đến khi nhận được trang web (chỉ tính kết quả của yêu cầu HTTP GET đầu tiên)?	7
2.3	Trang web này (server) có sử dụng gzip để nén dữ liệu khi gửi đi không? Nó có ghi cookie không?	8
2.4	Gói tin số 26 và 27 đang muốn làm gì? Theo bạn, tại sao lại phát sinh gói 26 và 27? . . .	9
2.5	Tại gói tin thứ 37 là một gói tin DNS query khác, cho domain là us.i1.yimg.com. Tại sao client lại cần hỏi địa chỉ IP cho domain này? Trong khi gói tin số 26 đã làm việc này rồi phải không?	11
2.6	Tại gói 42, bạn sẽ thấy một câu lệnh HTTP “Get”, và trong gói 48 là một câu lệnh HTTP “Get” mới. Tại sao hệ thống không cần một yêu cầu DNS khác trước câu lệnh get thứ hai? (Nhấp vào gói 42 và nhìn vào cửa sổ giữa. Mở rộng dòng có tiêu đề “Hypertext Transfer Protocol” và đọc dòng “Host:”. So sánh dòng đó với dòng “Host:” của gói 48)	12
2.7	Nhấp chuột phải lên gói tin thứ 31 => Follow => TCP Stream.	14
3	DHCP	19
3.1	Lọc gói tin theo giao thức DHCP	19
3.2	Giao thức tầng transport của DHCP	19
3.3	Địa chỉ tầng link của host được cấp IP	20
3.4	Cho biết thông tin IP nguồn, Port nguồn và IP đích, Port đích của 4 gói tin (Discover/Offer/Request/ACK) trong đợt xin cấp IP đầu tiên	21
3.5	Chỉ ra điểm khác biệt giữa các giá trị trong gói tin DHCP Discover và DHCP Request . .	23
3.6	Cho biết giá trị Transaction-ID của 4 gói tin (Discover/Offer/Request/ACK) trong đợt xin cấp IP đầu tiên, giá trị Transaction-ID của 2 gói tin (Request/ACK) trong đợt xin cấp IP lần hai. Mục đích sử dụng của transaction-ID là gì?	23
3.7	Cho biết ý nghĩa trường thông tin lease-time, giá trị của trường này trong gói tin bắt được là bao nhiêu?	25
3.8	Trong gói tin DHCP bắt được, hãy chỉ ra trường giá trị cho biết có thông tin DHCP relay agent hay không? Nếu có hãy cho biết IP của DHCP relay agent.	25

4 ICMP	27
4.1 MSS (Maximum Segment Size) là gì? Liên quan gì đến MTU (Maximum Transmission Unit)?	27
4.2 Cho biết MTU trên card mạng kết nối internet của máy bạn là bao nhiêu (có chụp hình)?	28
4.3 Kích thước MTU là 1500, cho biết kích thước của các phần của gói ICMP tại tầng mạng cơ bản (network) là bao nhiêu? (cho biết cách tính toán của bạn để có kết quả của từng phần)	28
4.4 Kích thước MTU là 1500, cho biết kích thước của các phần của gói ICMP tại tầng Data Link cơ bản là bao nhiêu? (cho biết cách tính toán của bạn để có kết quả của từng phần)	29
4.5 Câu lệnh ping -i 5 <ip của default gateway> và ping -l 33 <ip của default gateway> . . .	31
4.6 Lệnh ping -l 1472 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu?	33
4.7 Lệnh ping -l 1475 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu? Gói tin khác gì so với câu 6 bạn đã bắt ở trên và giải thích sự khác nhau đó? .	34
4.8 Lệnh ping -f -l 1475 192.168.1.8. Sau khi bắt gói tin bạn thấy vấn đề gì và giải thích kết quả mà bạn có được?	35
Tài liệu tham khảo	37

1 Giới thiệu

Trong thời đại công nghệ thông tin phát triển vượt bậc, việc phân tích dữ liệu mạng đóng vai trò quan trọng trong việc đảm bảo hiệu quả hoạt động và bảo mật của các hệ thống mạng. Để hiểu rõ cách thức giao tiếp giữa các thiết bị trong mạng, việc nghiên cứu và phân tích gói tin là bước không thể thiếu. Các giao thức như HTTP, DHCP, và ICMP thường được sử dụng rộng rãi và đóng vai trò cốt lõi trong quá trình trao đổi dữ liệu và quản lý mạng.

Đồ án này được thực hiện với mục tiêu sử dụng công cụ Wireshark để phân tích hoạt động của ba giao thức quan trọng:

- **HTTP (Hypertext Transfer Protocol):** Giao thức tầng ứng dụng phổ biến trong việc truyền tải dữ liệu web.
- **DHCP (Dynamic Host Configuration Protocol):** Giao thức cấu hình tự động địa chỉ IP và các thông số mạng cho các thiết bị trong mạng.
- **ICMP (Internet Control Message Protocol):** Giao thức hỗ trợ chẩn đoán và xử lý lỗi trong giao tiếp mạng.

Dưới sự hướng dẫn tận tình của *Ths. Huỳnh Thụy Bảo Trân* và *Ths. Chung Thùy Linh*, nhóm chúng tôi đã hoàn thành đồ án với việc *thu thập và phân tích gói tin của ba giao thức HTTP, DHCP, và ICMP thông qua Wireshark*. Báo cáo này sẽ trình bày chi tiết *Thông tin, bảng phân công nhóm, và từng phần phân tích gói tin*.

Chúng tôi xin gửi lời cảm ơn chân thành đến những người bạn, người anh, người chị vì sự hỗ trợ quý báu trong suốt quá trình thực hiện đồ án, cũng như đến các hai giảng viên hướng dẫn đã cung cấp nền tảng kiến thức vững chắc giúp chúng tôi hoàn thành tốt nhiệm vụ này. Trong quá trình làm đồ án, nhóm chúng tôi vừa học kiến thức mới cũng như lần đầu được tiếp xúc trong việc phân tích gói tin nên không thể thiếu những sai sót và hạn chế hy vọng giảng viên có thể giúp đỡ chúng tôi nhận ra và cải thiện. Xin cảm ơn.

1.1 Thông tin thành viên

STT	MSSV	Họ và tên	Email
1	23120233	Nguyễn Lê Hữu Điền	gaolua217@gmail.com
2	23120262	Tổng Dương Thái Hòa	tdthoa.hry@gmail.com
3	23120264	Nguyễn Phúc Hoàng	nphuchoang.itus@gmail.com

Bảng 1: Thông tin thành viên

1.2 Phân chia công việc

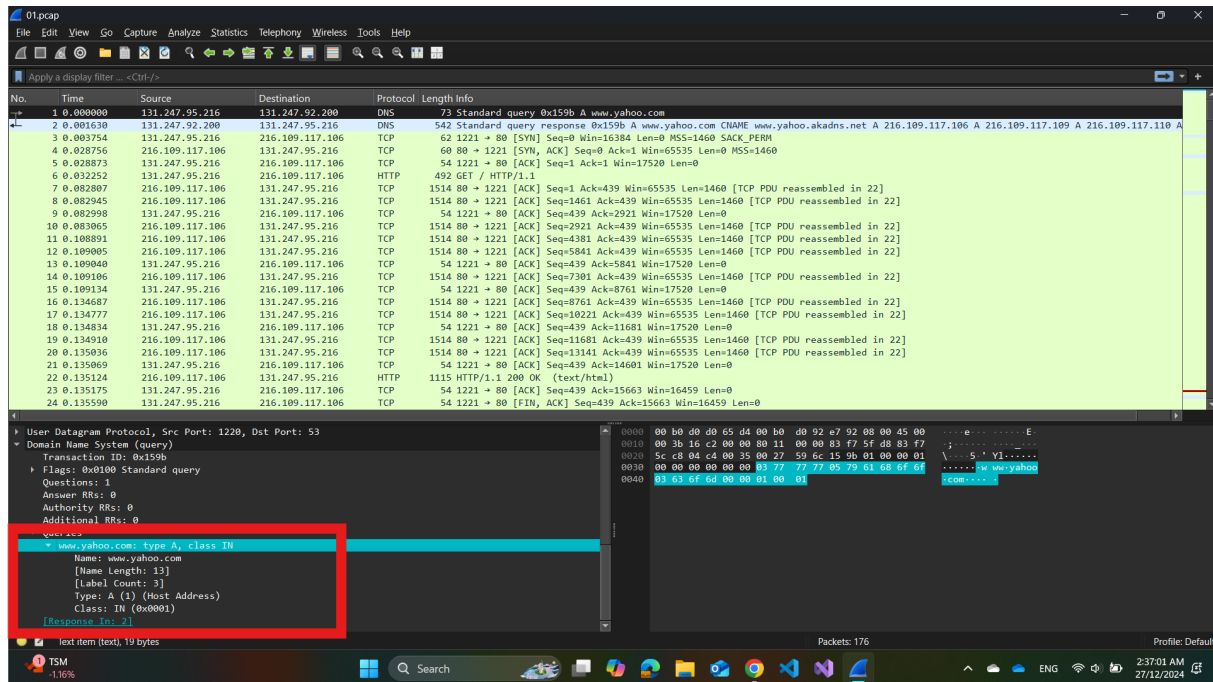
STT	Yêu cầu	Mức độ hoàn thành	Thực hiện
PHÂN TÍCH			
1.1	Thực hiện câu 1: HTTP	100%	Thái Hòa
1.2	Thực hiện câu 2: DHCP	100%	Phúc Hoàng
1.3	Thực hiện câu 3: ICMP	100%	Hữu Điền
BÁO CÁO			
2.1	Giới thiệu	100%	Thái Hòa
2.2	HTTP	100%	Thái Hòa
2.3	DHCP	100%	Phúc Hoàng
2.4	ICMP	100%	Hữu Điền
4.6	Format lại báo cáo	100%	Thái Hòa

Bảng 2: Bảng Phân chia công việc

2 HTTP

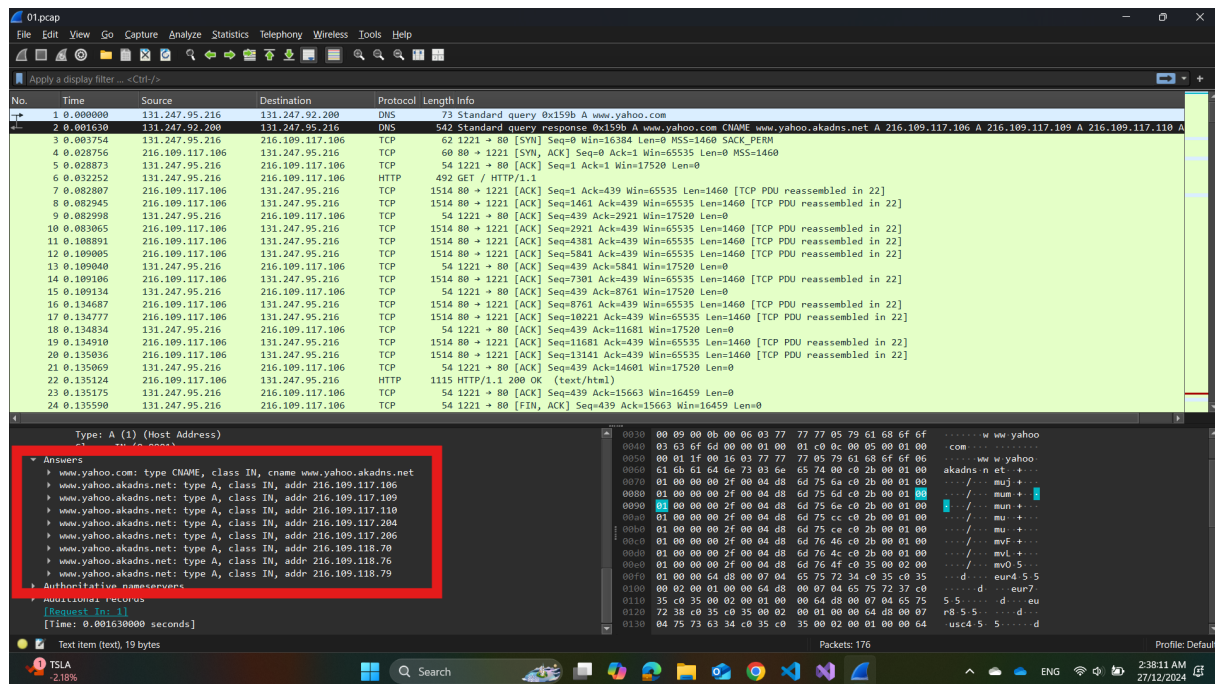
Ghi chú: Những vùng khoanh **màu đỏ** là các vị trí lấy thông tin của yêu cầu.

2.1 Nhìn vào những gói đầu tiên, máy client đang truy vấn IP của một tên miền. Cho biết tên miền của website là gì? Cho biết hai địa chỉ IPs của website này?



Hình 1: Tên địa chỉ website

Tên miền của website là **www.yahoo.com**

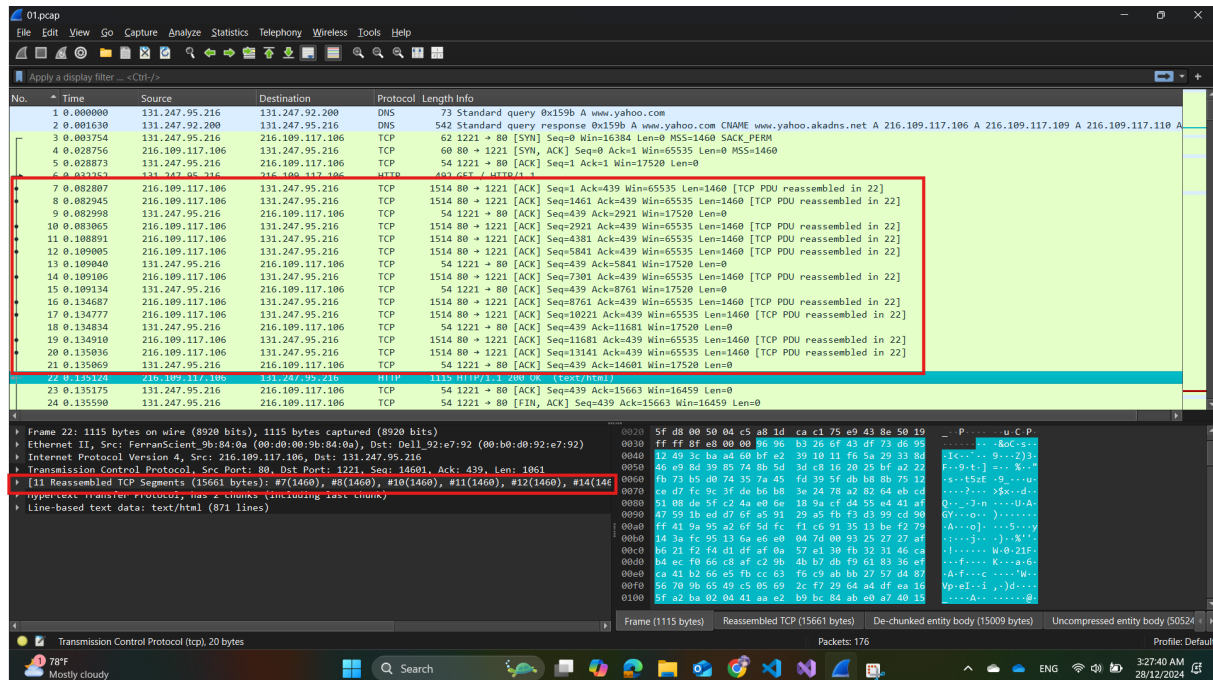


Hình 2: Địa chỉ IPs

Hai địa chỉ IPs của website:

- **216.109.117.106**
- **216.109.117.109**

2.2 Có bao nhiêu gói tin (packets) mà Server đã gửi cho máy client tính từ lúc bắt đầu đến khi nhận được trang web (chỉ tính kết quả của yêu cầu HTTP GET đầu tiên)?



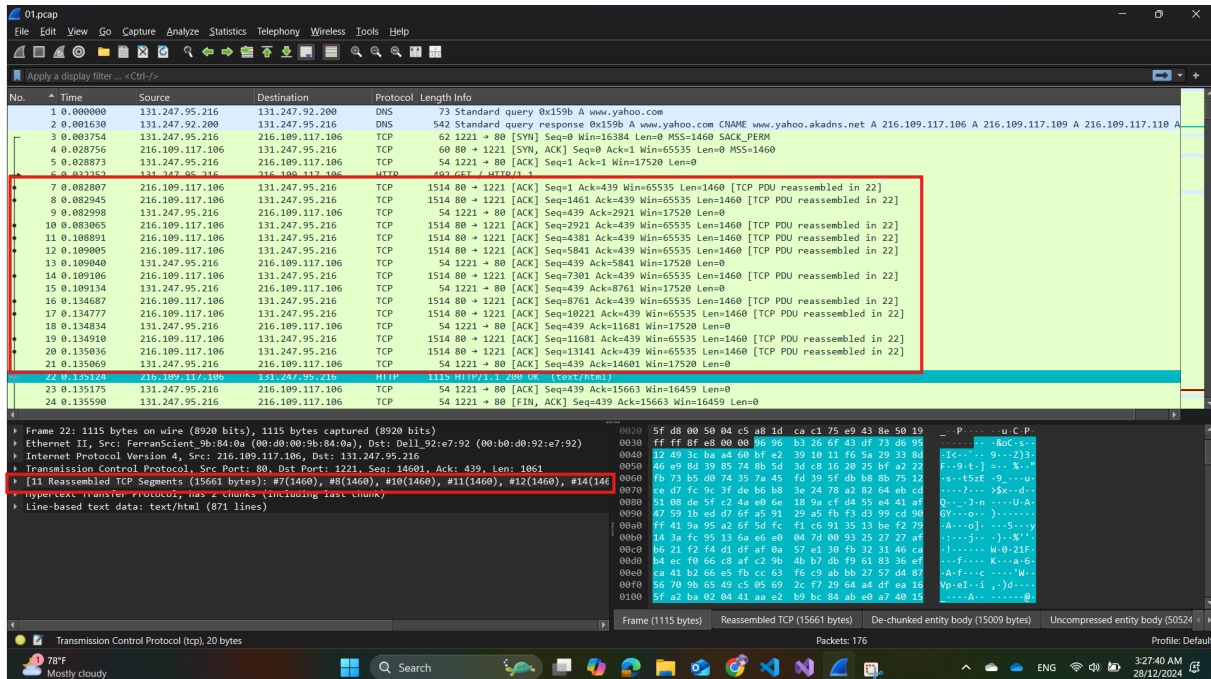
Hình 3: Số gói tin Server đã gửi cho Client

Yêu cầu HTTP GET đầu tiên (dòng 6): Client 131.247.95.216 gửi gói tin HTTP GET đến Server 216.109.117.106. Số packets mà Server đã gửi cho Client để đáp ứng yêu cầu HTTP GET đầu tiên: **Tổng cộng: 11 gói tin**, bao gồm:

- 10 gói TCP chứa dữ liệu: Mỗi gói có Len=1460 (dòng 7, 8, 10, 11, 12, 14, 16, 17, 19, 20).
- 1 gói chứa thông điệp HTTP hoàn chỉnh (HTTP/1.1 200 OK): Gói cuối cùng với Len=1115 (dòng 22).

Lưu ý không tính các gói liên quan đến quá trình bắt tay ba bước (TCP 3-way handshake): Gói SYN (dòng 3), SYN-ACK (dòng 4), và ACK (dòng 5).

2.3 Trang web này (server) có sử dụng gzip để nén dữ liệu khi gửi đi không? Nó có ghi cookie không?



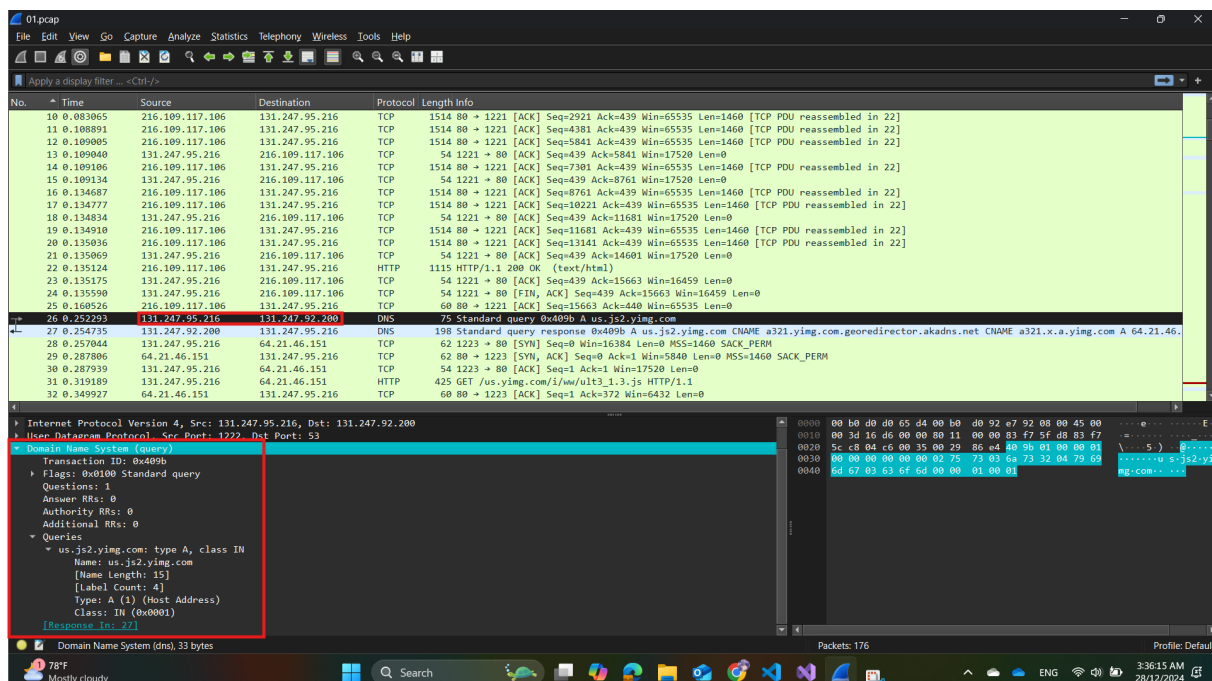
Hình 4: Content-Encoding

Content-Encoding (gzip): Trang web này **có** sử dụng gzip để nén dữ liệu trước khi gửi đi, giúp giảm dung lượng và tăng tốc độ truyền tải. Trang web **có** ghi cookie. Cụ thể hai cookie được ghi lại như sau:

- **Set-Cookie:** `FPB=ol1uquj7e125v8pe; expires=Thu, 01 Jun 2006 19:00:00 GMT; path=/; domain=www.yahoo.com\r\n`
 - **FPB=ol1uquj7e125v8pe:** Hỗ trợ theo dõi người dùng nhận dạng phiên làm việc, theo dõi người dùng, hoặc lưu trạng thái chỉ trong phạm vi domain `www.yahoo.com`.
 - **expires=Thu, 01 Jun 2006 19:00:00 GMT:** Cookie có thời hạn (*persistent cookie*) sẽ được lưu trên đĩa và tồn tại qua các lần mở trình duyệt khác nhau, sẽ hết hạn vào thời gian được chỉ định (01 tháng 6 năm 2006, lúc 19:00:00 UTC). Sau thời gian này, trình duyệt sẽ tự động xóa cookie.
 - **path=/:** Cookie này sẽ được gửi kèm trong tất cả các yêu cầu đến mọi đường dẫn (*path*) trong domain.
 - **domain=www.yahoo.com:** Cookie này chỉ áp dụng cho các yêu cầu đến domain `www.yahoo.com`. **Không áp dụng** cho các subdomain như `mail.yahoo.com` hoặc `finance.yahoo.com`.
 - **\r\n:** Đây là ký tự kết thúc dòng header trong giao thức HTTP, bắt buộc để chỉ rằng dòng **Set-Cookie** đã kết thúc và chuyển sang dòng header khác.
- **Set-Cookie:** `D=_ylh=X3oDMTFmdXFnaZJsBF9TAzI3MTYxNDkEcG1kAzExNDcxMtc5N TQEdGVzdAMwBHRtcGwDaW5kZXgtY3Nz; path=/; domain=.yahoo.com\r\n`

- **Cookie D:** Dùng để theo dõi người dùng trong toàn bộ hệ thống Yahoo (bao gồm các subdomain).
- Không có thời hạn (*expires không xuất hiện*), nên đây là session cookie. Trình duyệt sẽ xóa nó khi phiên làm việc (*session*) kết thúc hoặc khi người dùng đóng trình duyệt.
- **path=/:** Tương tự cookie trước, cookie này áp dụng cho tất cả các đường dẫn trong domain được chỉ định.
- **domain=.yahoo.com:** Cookie này áp dụng cho toàn bộ domain .yahoo.com và các subdomain của nó.
- **\r\n:** Đây là ký tự kết thúc dòng header trong giao thức HTTP, bắt buộc để chỉ rằng dòng Set-Cookie đã kết thúc và chuyển sang dòng header khác.

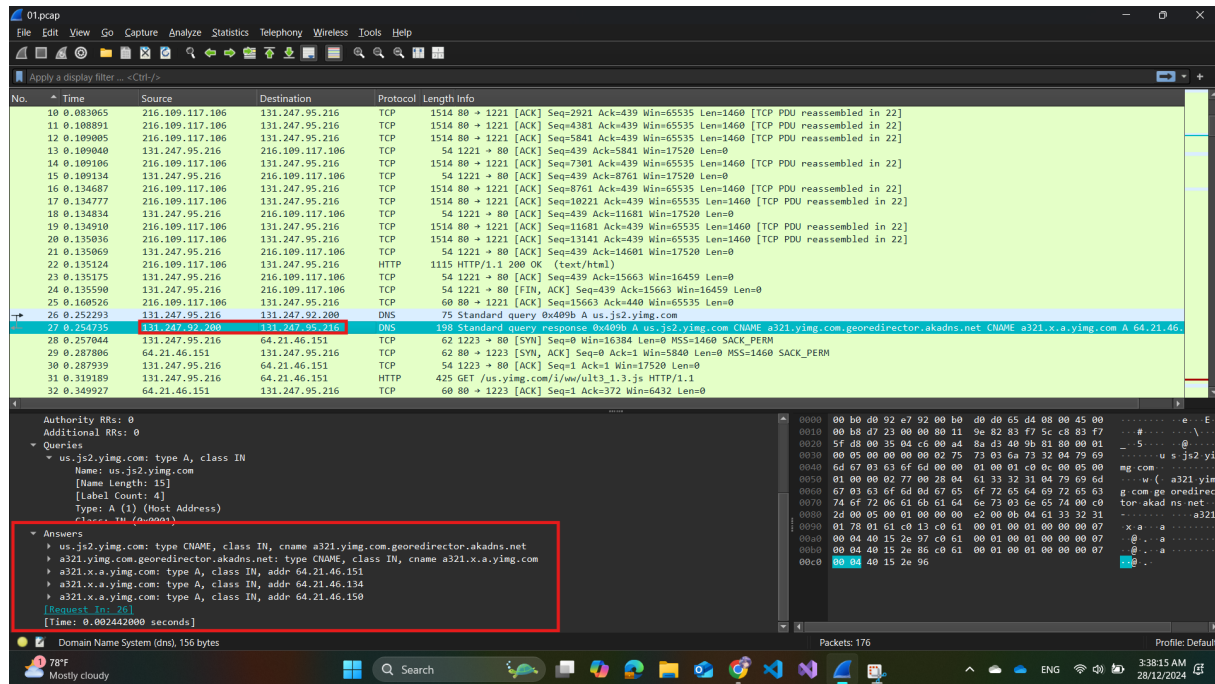
2.4 Gói tin số 26 và 27 đang muốn làm gì? Theo bạn, tại sao lại phát sinh gói 26 và 27?



Hình 5: Gói tin số 26

Gói tin 26:

- Máy tính (Client) tại địa chỉ IP **131.247.95.216** thực hiện truy vấn DNS đến máy chủ (Server) tại địa chỉ IP **131.247.92.200** để phân giải tên miền phụ **us.js2.yimg.com** thành địa chỉ IP.
- Mục đích là để trình duyệt có thể tải tài nguyên từ miền phụ này.



Hình 6: Gói tin số 27

Gói tin 27: Máy chủ DNS tại địa chỉ IP 131.247.92.200 phản hồi kết quả truy vấn DNS với thông tin:

- **us.js2.yimg.com** là một CNAME (Canonical Name Record) trỏ đến **a321.yimg.com.georedirector.akadns.net**.
- CNAME này được phân giải thành ba địa chỉ IPs:
 - 64.21.46.151
 - 64.21.46.134
 - 64.21.46.150
- Mục đích là để cung cấp địa chỉ IP để trình duyệt kết nối và tải tài nguyên.

Tại sao phát sinh gói tin 26 và 27?

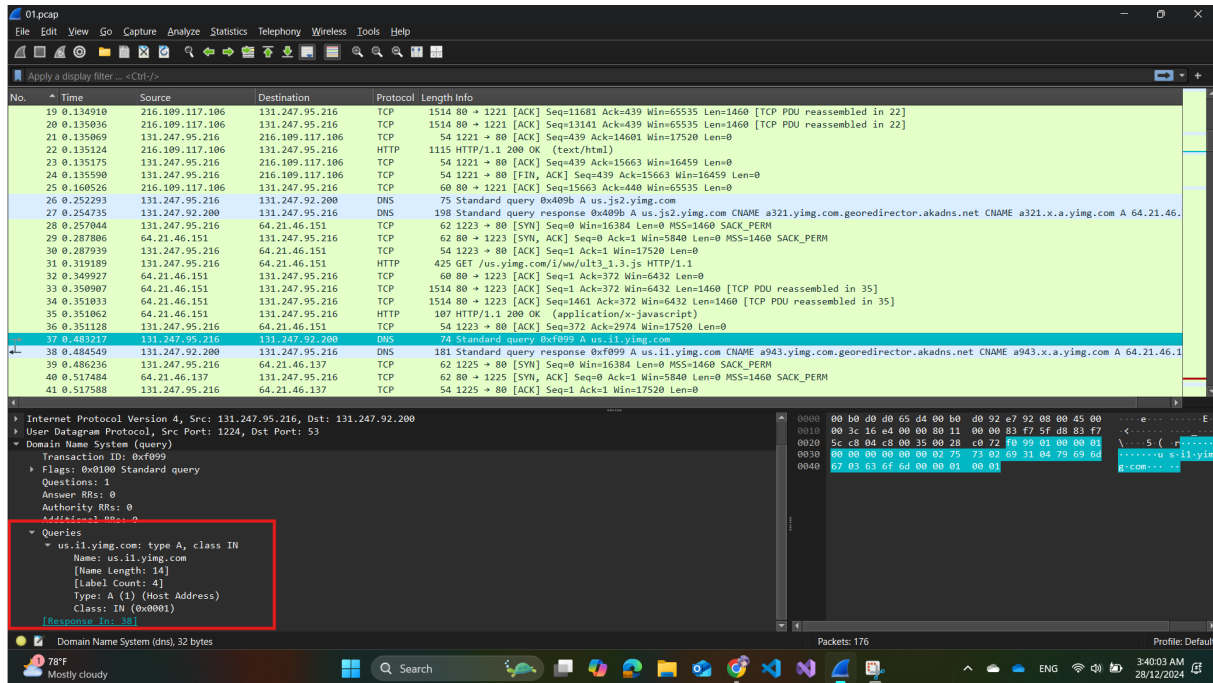
Nguyên nhân phát sinh gói tin 26:

- Khi trình duyệt tải một trang web, trang này yêu cầu tài nguyên bổ sung (hình ảnh, tập lệnh, CSS) từ miền phụ **us.js2.yimg.com**. Trước khi có thể giao tiếp với tên miền này, nó phải thực hiện truy vấn DNS để biết được địa chỉ IP tương ứng.
- Vì miền phụ này chưa được phân giải trước đó, trình duyệt cần gửi truy vấn DNS để biết địa chỉ IP của miền.

Nguyên nhân phát sinh gói tin 27 là do máy chủ DNS phản hồi truy vấn DNS của gói tin 26 bằng cách cung cấp:

- Thông tin CNAME (**us.js2.yimg.com** → **a321.yimg.com.georedirector.akadns.net**).
- Địa chỉ IP tương ứng của miền phụ để hoàn thành quá trình phân giải tên.

2.5 Tại gói tin thứ 37 là một gói tin DNS query khác, cho domain là **us.il.yimg.com**. Tại sao client lại cần hỏi địa chỉ IP cho domain này? Trong khi gói tin số 26 đã làm việc này rồi phải không?



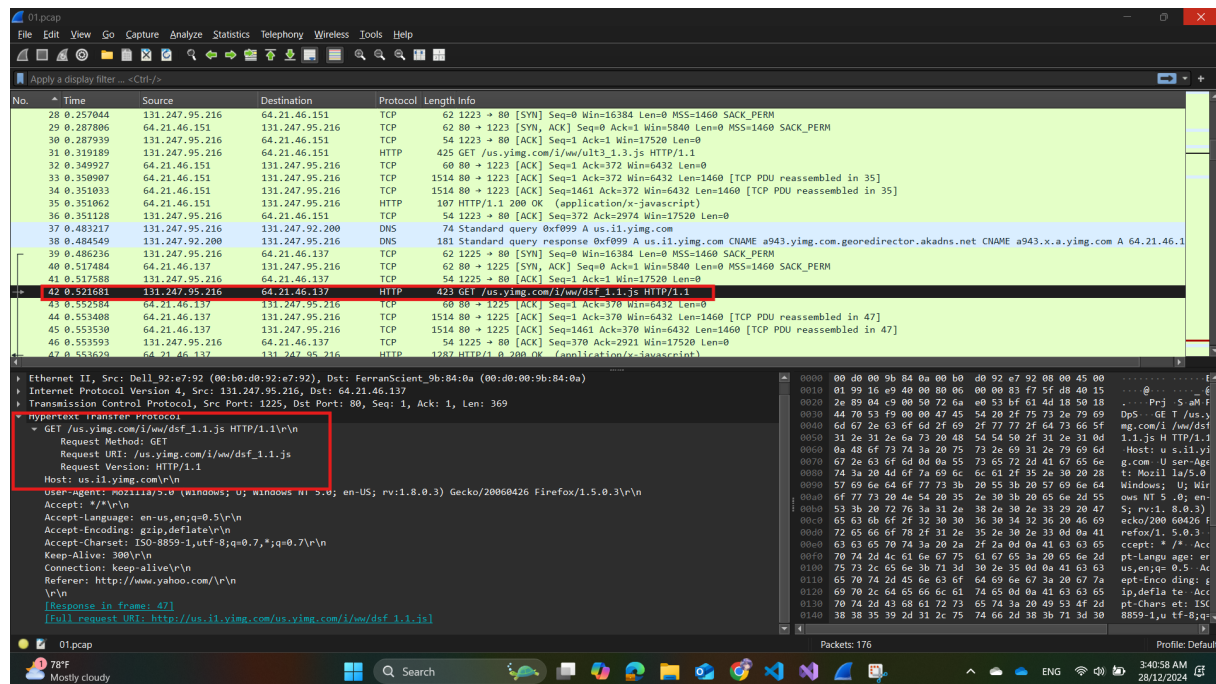
Hình 7: Gói tin số 37

Không phải, gói tin số 26 không thực hiện phân giải tên miền **us.il.yimg.com** giống gói tin số 37 mà là **us.js2.yimg.com**. Đây là hai tên miền hoàn toàn khác nhau.

Lý do client cần hỏi địa chỉ IP cho tên miền mới (**us.il.yimg.com**):

- **Tài nguyên từ nhiều tên miền khác nhau:** Khi ứng dụng hoặc trình duyệt tải nội dung từ một trang web, nó có thể yêu cầu tài nguyên từ nhiều tên miền phụ hoặc máy chủ khác nhau. Vì vậy, nếu cần tải tài nguyên từ một tên miền mới (**us.il.yimg.com**), trình duyệt phải thực hiện thêm một truy vấn DNS để phân giải tên miền này thành địa chỉ IP.
- **Không thể dùng lại kết quả DNS từ gói 26:** Truy vấn DNS được thiết kế để giải quyết từng tên miền cụ thể. Vì **us.js2.yimg.com** và **us.il.yimg.com** là hai tên miền khác nhau, mỗi tên miền cần một truy vấn DNS riêng biệt.

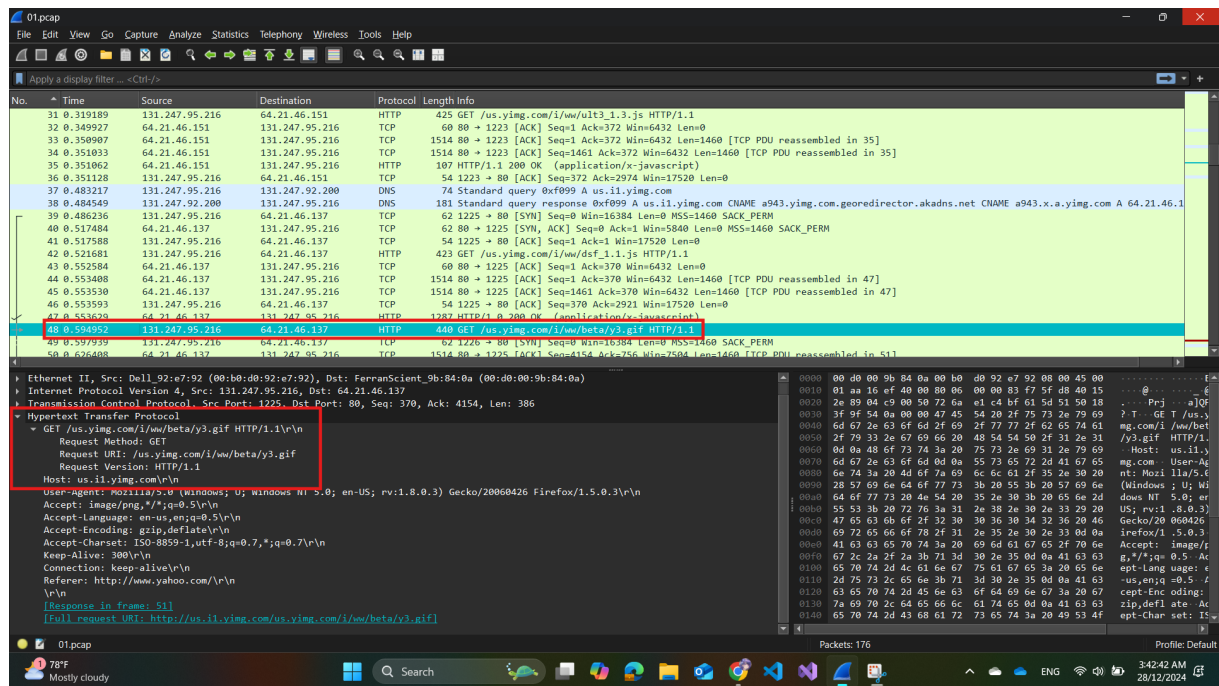
2.6 Tại gói 42, bạn sẽ thấy một câu lệnh HTTP “Get”, và trong gói 48 là một câu lệnh HTTP “Get” mới. Tại sao hệ thống không cần một yêu cầu DNS khác trước câu lệnh get thứ hai? (Nhấp vào gói 42 và nhìn vào cửa sổ giữa. Mở rộng dòng có tiêu đề “Hypertext Transfer Protocol” và đọc dòng “Host:”. So sánh dòng đó với dòng “Host:” của gói 48)



Hình 8: Gói tin số 42

Câu lệnh HTTP GET trong gói 42:

- Host: **us.il.yimg.com**.
- Trước khi gửi gói 42, hệ thống đã thực hiện truy vấn DNS (gói 37) để phân giải tên miền này thành địa chỉ IP, và kết quả được lưu trong DNS cache của client.



Hình 9: Gói tin số 48

Câu lệnh HTTP GET trong gói 48:

- Host: **us.il.yimg.com**.
- Do hệ thống đã phân giải tên miền **us.il.yimg.com** trước đó và lưu kết quả trong DNS cache, client không cần gửi lại truy vấn DNS mới.

Sử dụng DNS cache:

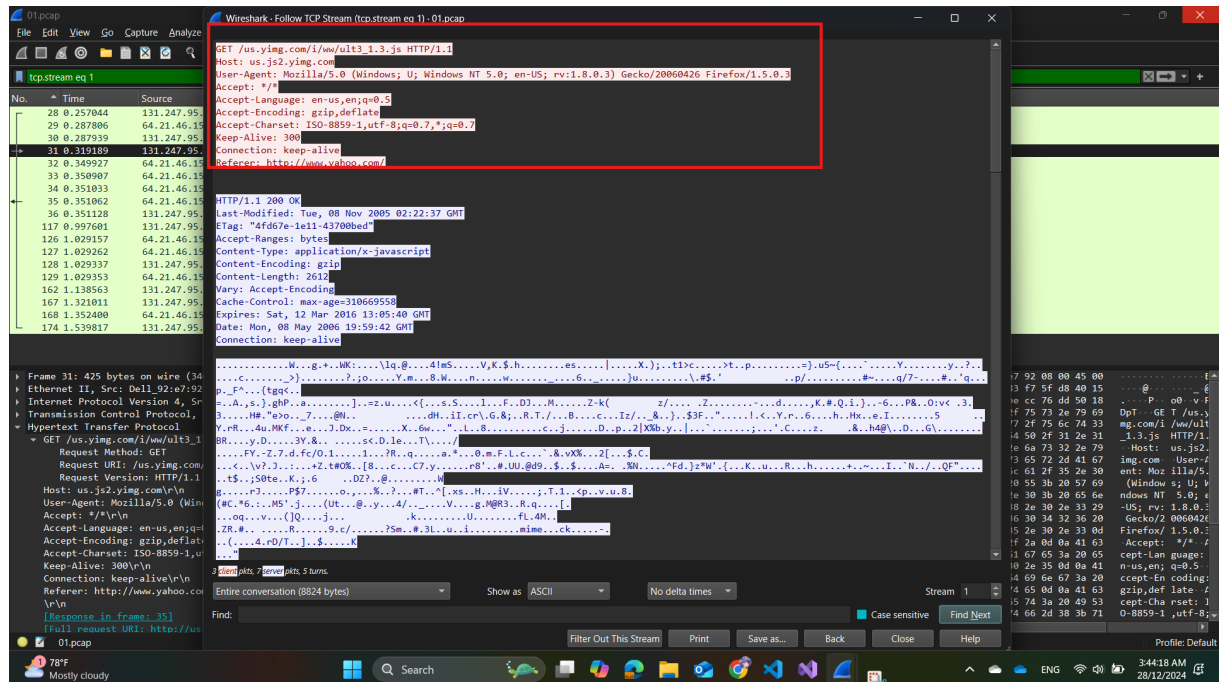
- **Lưu trữ địa chỉ IP sau truy vấn DNS:** Khi client thực hiện một truy vấn DNS và nhận được phản hồi, địa chỉ IP của tên miền sẽ được lưu vào DNS cache của hệ thống. Bên cạnh đó, thời gian lưu trữ phụ thuộc vào giá trị TTL (Time To Live) của bản ghi DNS.
- **Tái sử dụng thông tin từ DNS cache:** Nếu có yêu cầu tiếp theo đến cùng tên miền trong thời gian TTL còn hiệu lực, hệ thống sẽ sử dụng thông tin từ DNS cache, không cần gửi truy vấn DNS mới.

Tên miền không thay đổi:

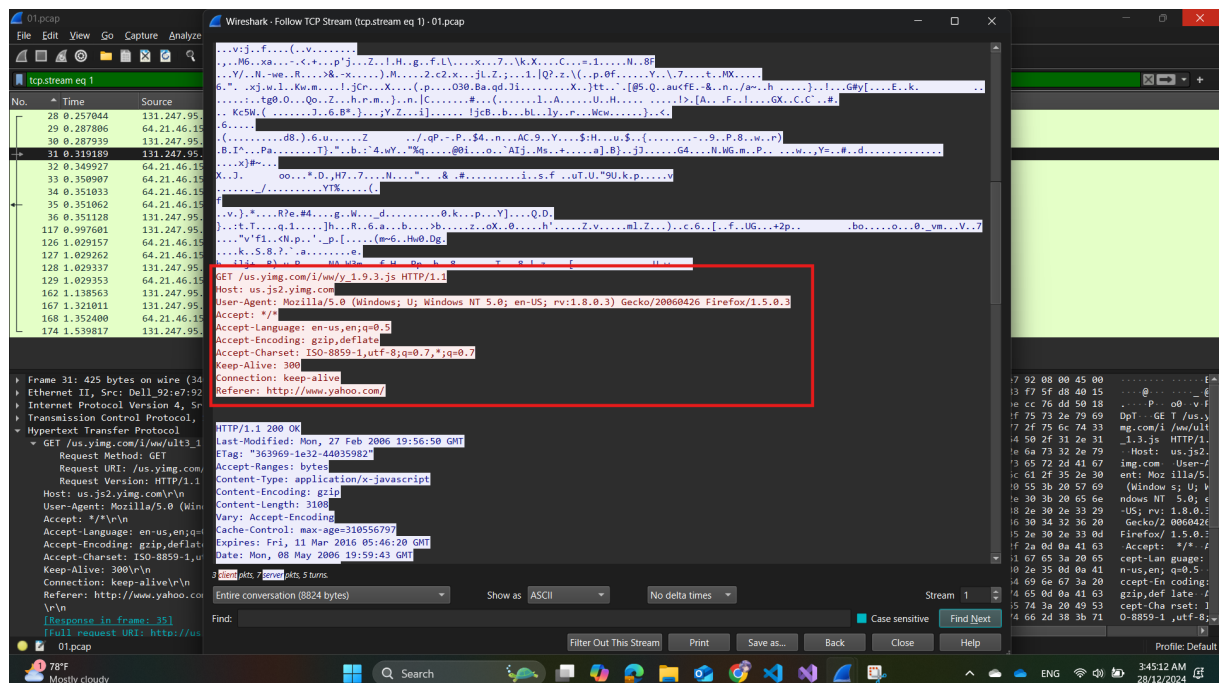
- Trường Host trong cả gói 42 và gói 48 đều chỉ ra rằng hai yêu cầu HTTP nhắm đến cùng một tên miền (**us.il.yimg.com**).
- Do đó, client không cần thực hiện phân giải DNS lại, giúp giảm độ trễ và tiết kiệm tài nguyên hệ thống.

2.7 Nhấp chuột phải lên gói tin thứ 31 => Follow => TCP Stream.

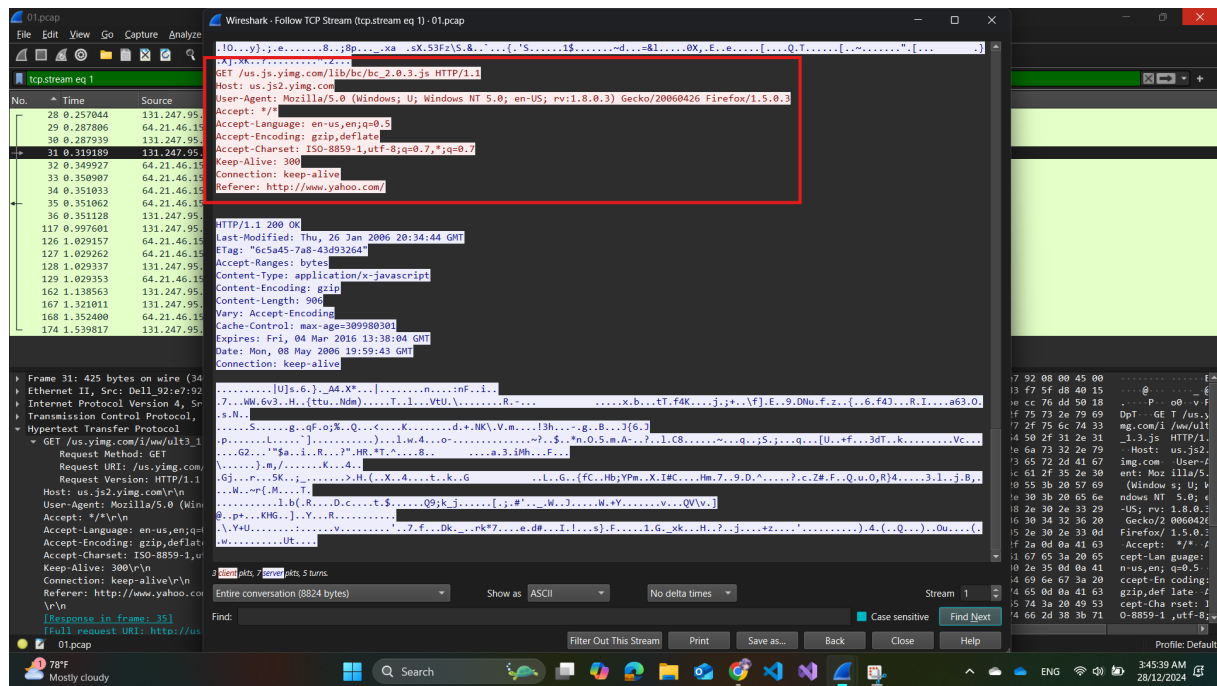
2.7.1 Cho biết có bao nhiêu HTTP request từ client gửi đi?



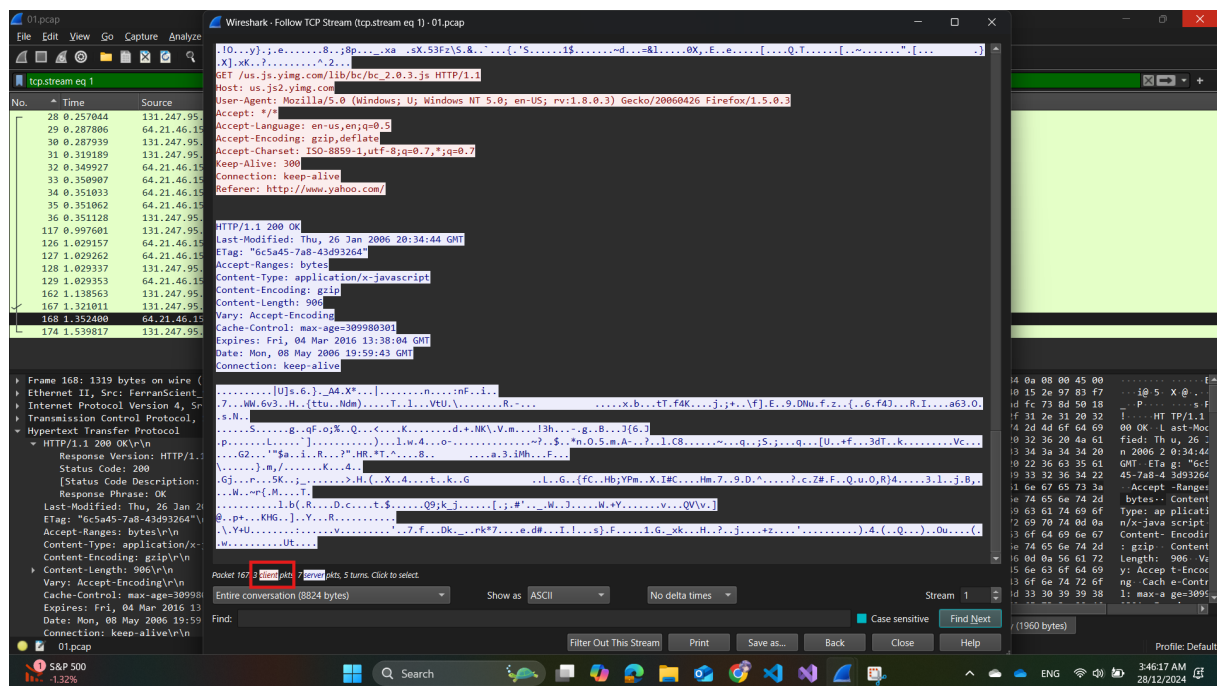
Hình 10: REQUEST 1



Hình 11: REQUEST 2



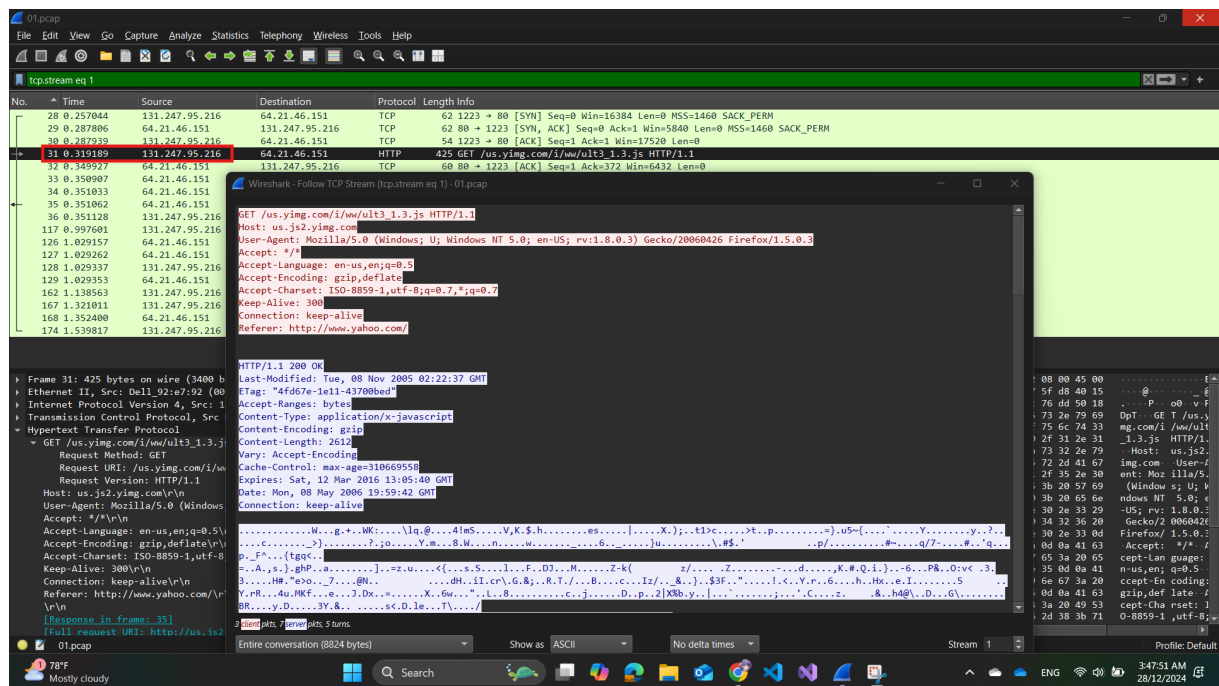
Hình 12: REQUEST 3



Hình 13: Tổng số REQUEST

Có 3 HTTP requests từ Client gửi đi.

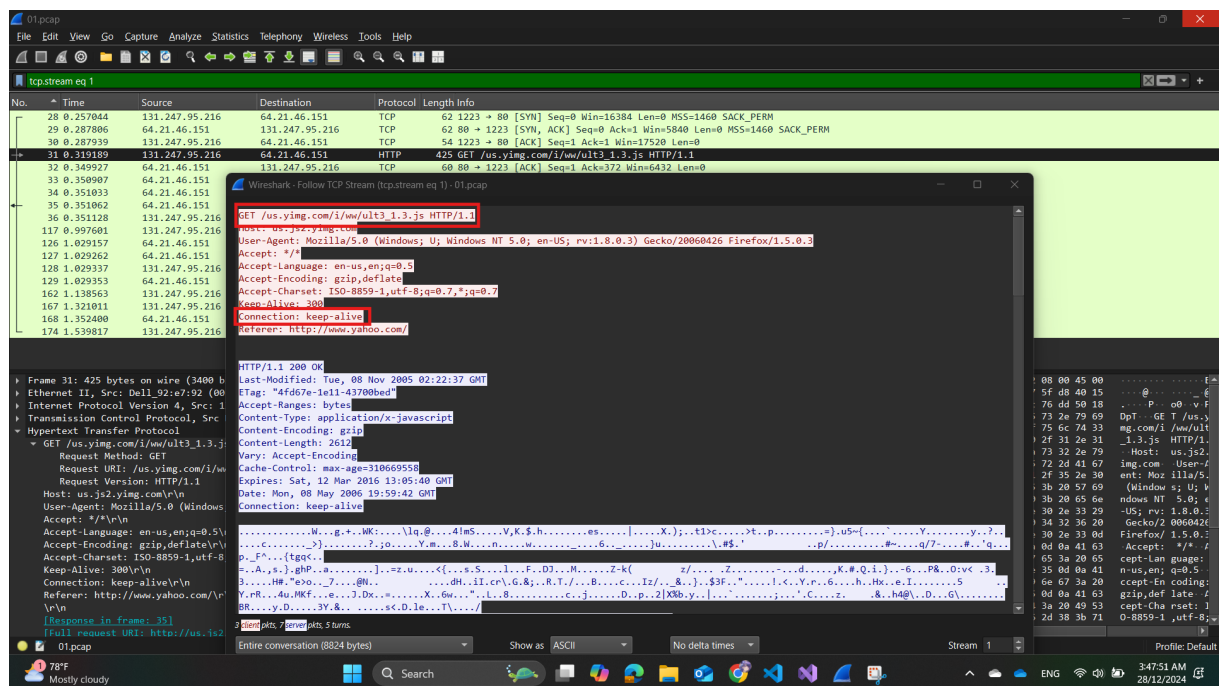
2.7.2 Cho biết IP của máy client?



Hình 14: Gói tin số 31

IP của máy Client là: **131.247.95.216**.

2.7.3 Cho biết loại HTTP connections là gì? (Non-persistent HTTP hay Persistent HTTP), tại sao?



Hình 15: HTTP Connections

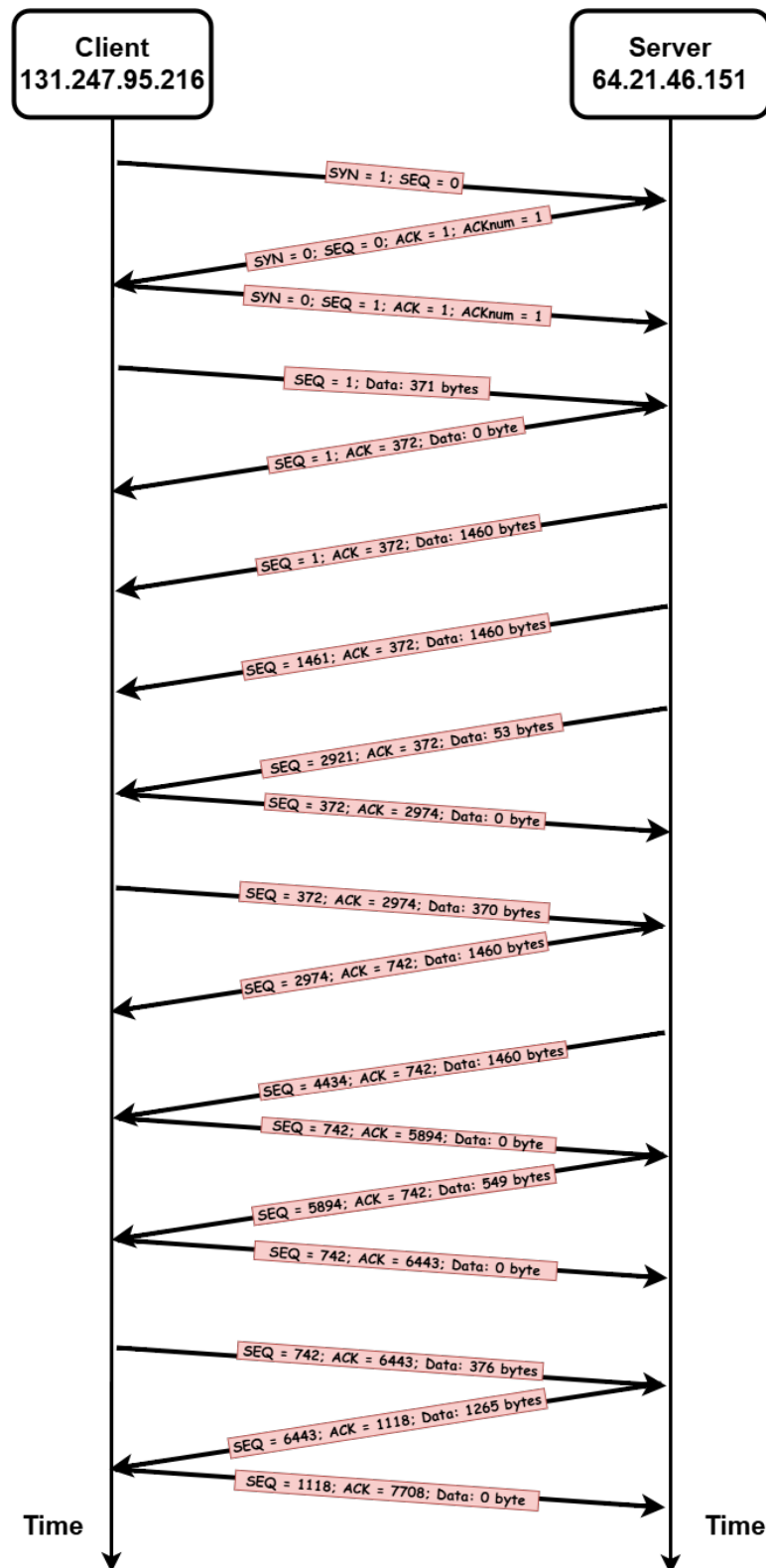
Sử dụng tiêu đề `Connection: keep-alive`:

- **Giữ kết nối mở:** Trong cả yêu cầu và phản hồi HTTP, tiêu đề `Connection: keep-alive` được sử dụng để yêu cầu máy chủ duy trì kết nối mở sau khi phản hồi hoàn tất. Điều này cho phép tái sử dụng kết nối TCP hiện tại cho các yêu cầu HTTP tiếp theo, giảm độ trễ do không cần thiết lập kết nối mới.
- **Mặc định trong HTTP/1.1:** Giao thức `HTTP/1.1` mặc định bật `Persistent Connections`. Kết nối sẽ chỉ được đóng nếu tiêu đề `Connection: close` xuất hiện trong yêu cầu hoặc phản hồi.

Lợi ích của `Connection: keep-alive`:

- **Hiệu suất tốt hơn:** Giảm số lượng thiết lập và đóng kết nối TCP, tiết kiệm tài nguyên hệ thống.
- **Tốc độ cao hơn:** Giảm độ trễ cho các yêu cầu HTTP liên tiếp trên cùng một kết nối.

2.7.4 Vẽ hình quá trình trao đổi gói tin giữa client và máy chủ bắt đầu từ gói tin TCP SYN đến khi kết thúc tcp stream filter.



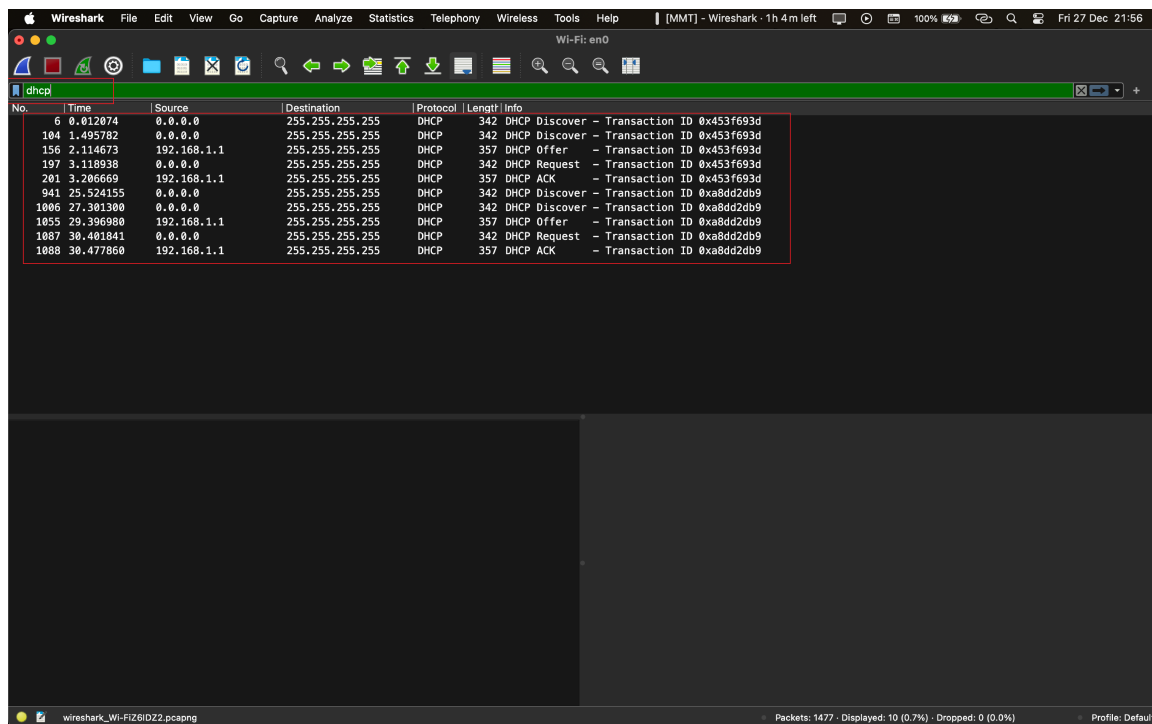
Hình 16: Quá trình trao đổi gói tin

3 DHCP

Ghi chú: Những vùng khoanh **màu đỏ** là các vị trí lấy thông tin của yêu cầu.

3.1 Lọc gói tin theo giao thức DHCP

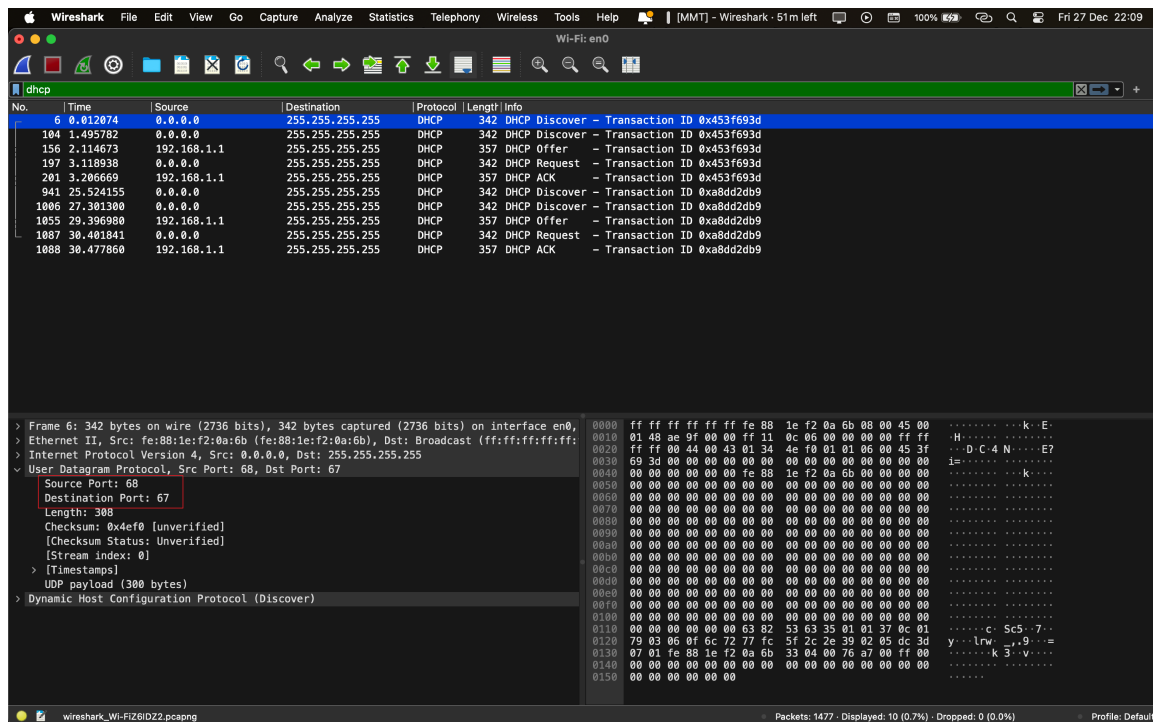
Tại bộ lọc, nhập `dhcp` để lọc các gói tin DHCP.



Hình 17: Lọc gói tin theo giao thức DHCP

3.2 Giao thức tầng transport của DHCP

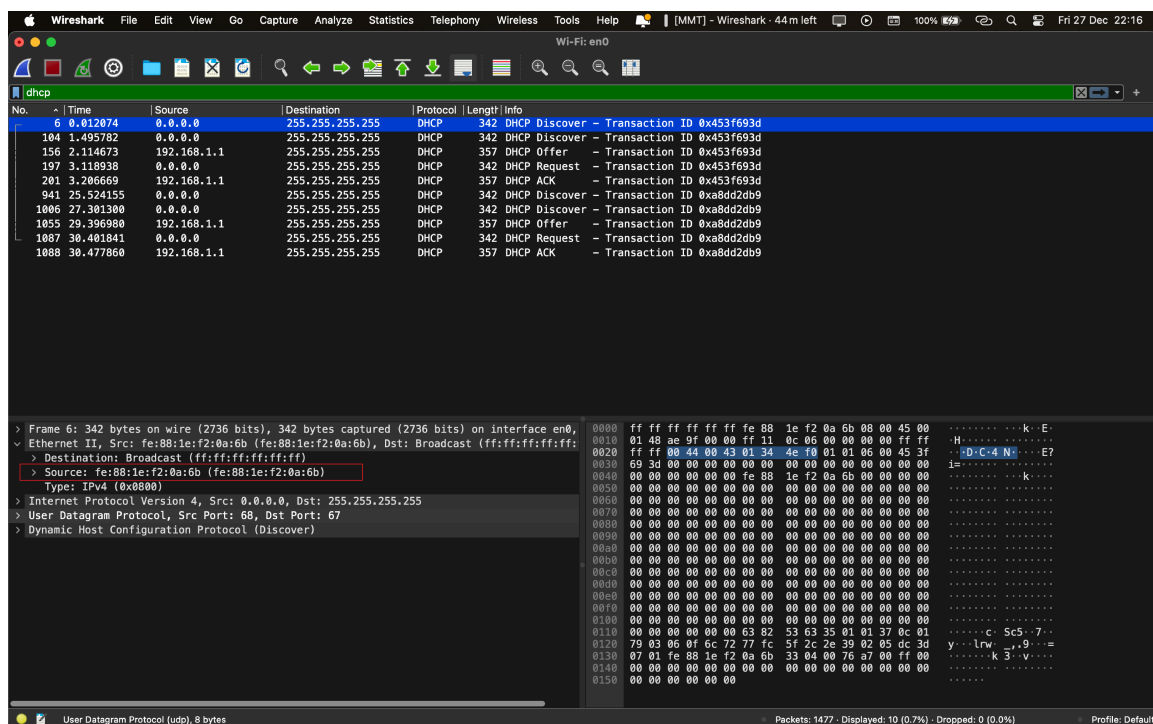
Gói tin DHCP sử dụng giao thức **UDP** tại tầng transport.



Hình 18: Gói tin DHCP sử dụng giao thức UDP

3.3 Địa chỉ tầng link của host được cấp IP

Địa chỉ MAC của host được cấp IP trong gói tin DHCP Discover: **fe:88:1e:f2:0a:6b**.

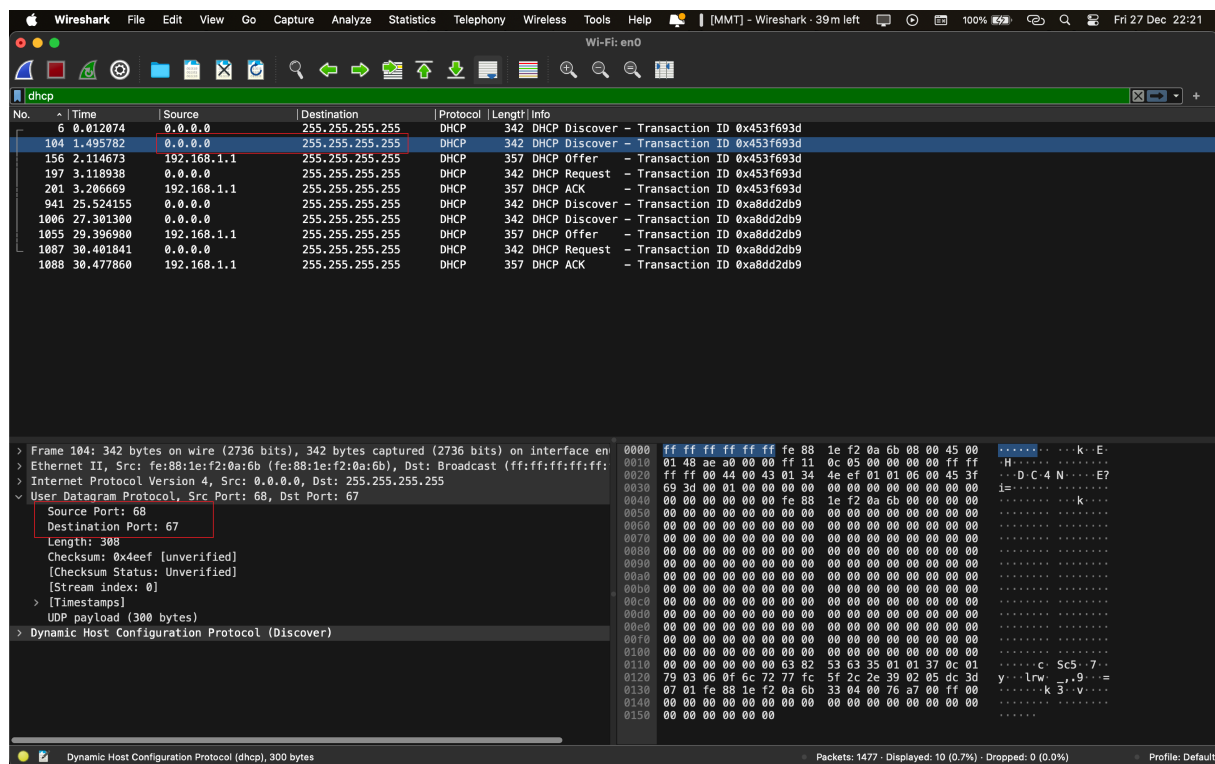


Hình 19: Địa chỉ MAC của host được cấp IP

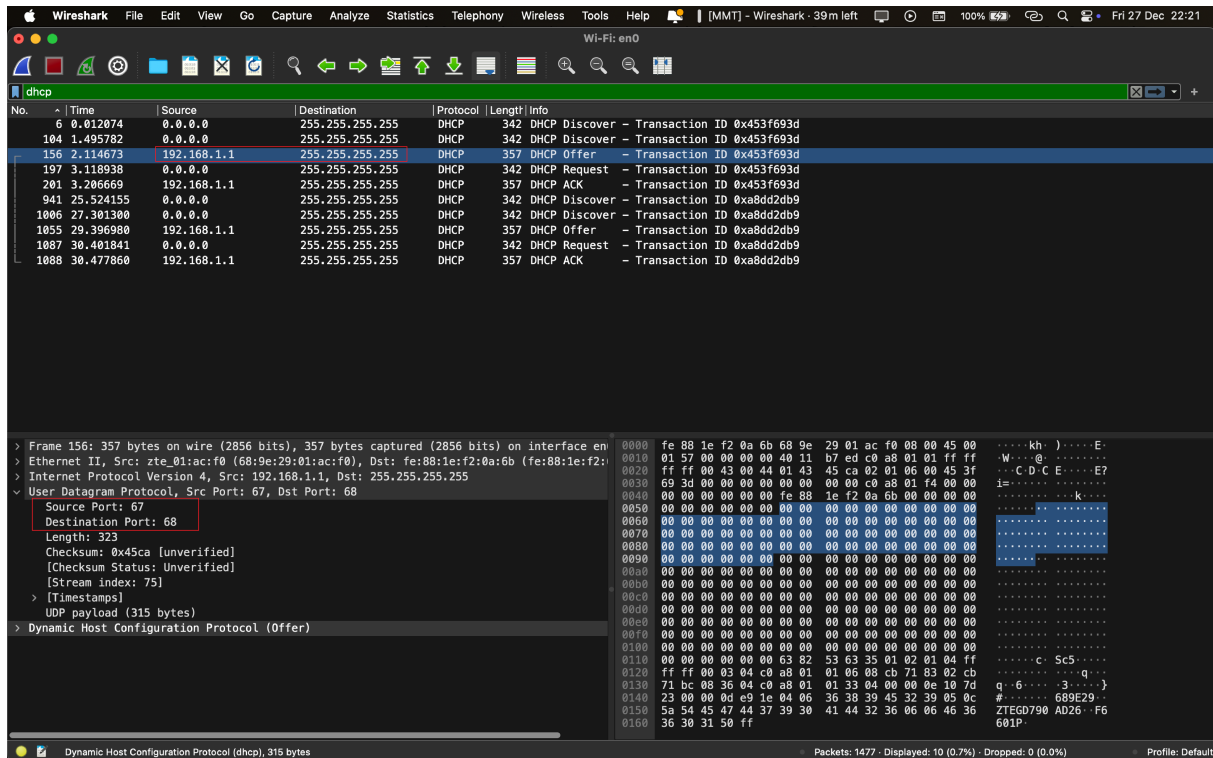
3.4 Cho biết thông tin IP nguồn, Port nguồn và IP đích, Port đích của 4 gói tin (Discover/Offer/ Request/ACK) trong đợt xin cấp IP đầu tiên

Gói tin	IP nguồn	Port nguồn	IP đích	Port đích
DHCP Discover	0.0.0.0	68	255.255.255.255	67
DHCP Offer	192.168.1.1	67	255.255.255.255	68
DHCP Request	0.0.0.0	68	255.255.255.255	67
DHCP ACK	192.168.1.1	67	255.255.255.255	68

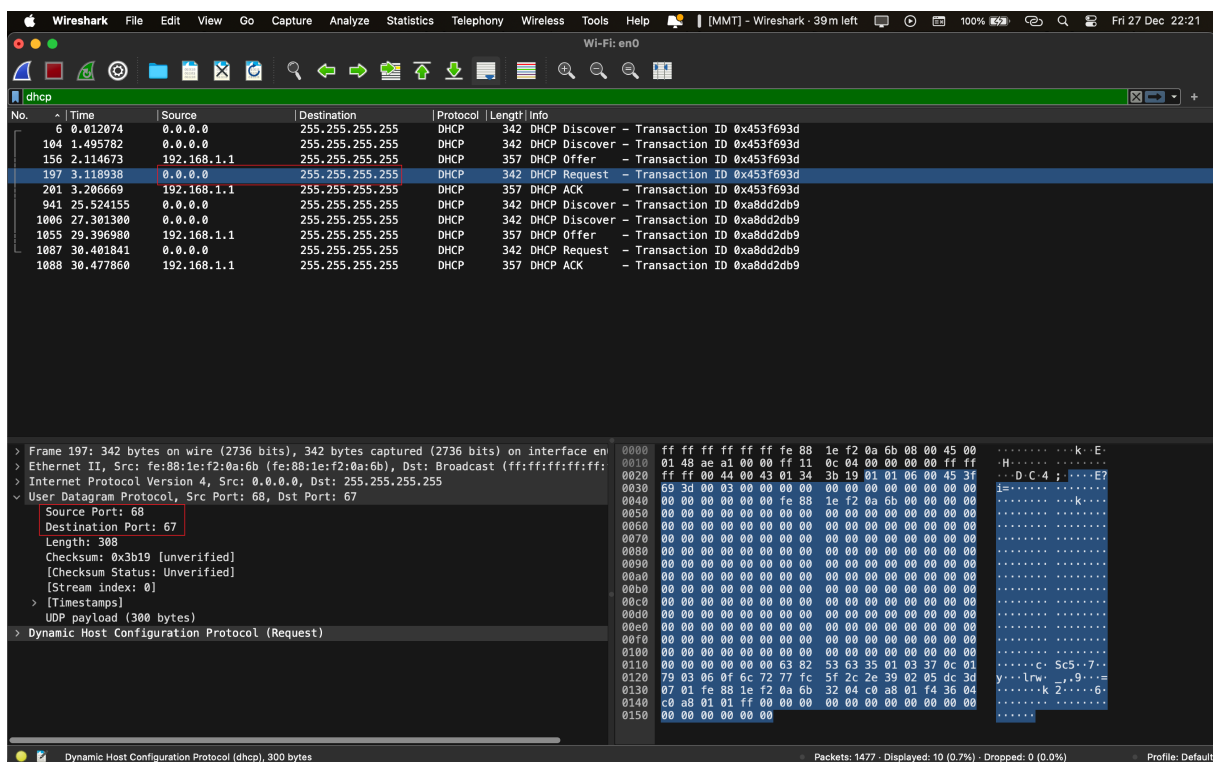
Bảng 3: Thông tin IP và Port của các gói tin DHCP



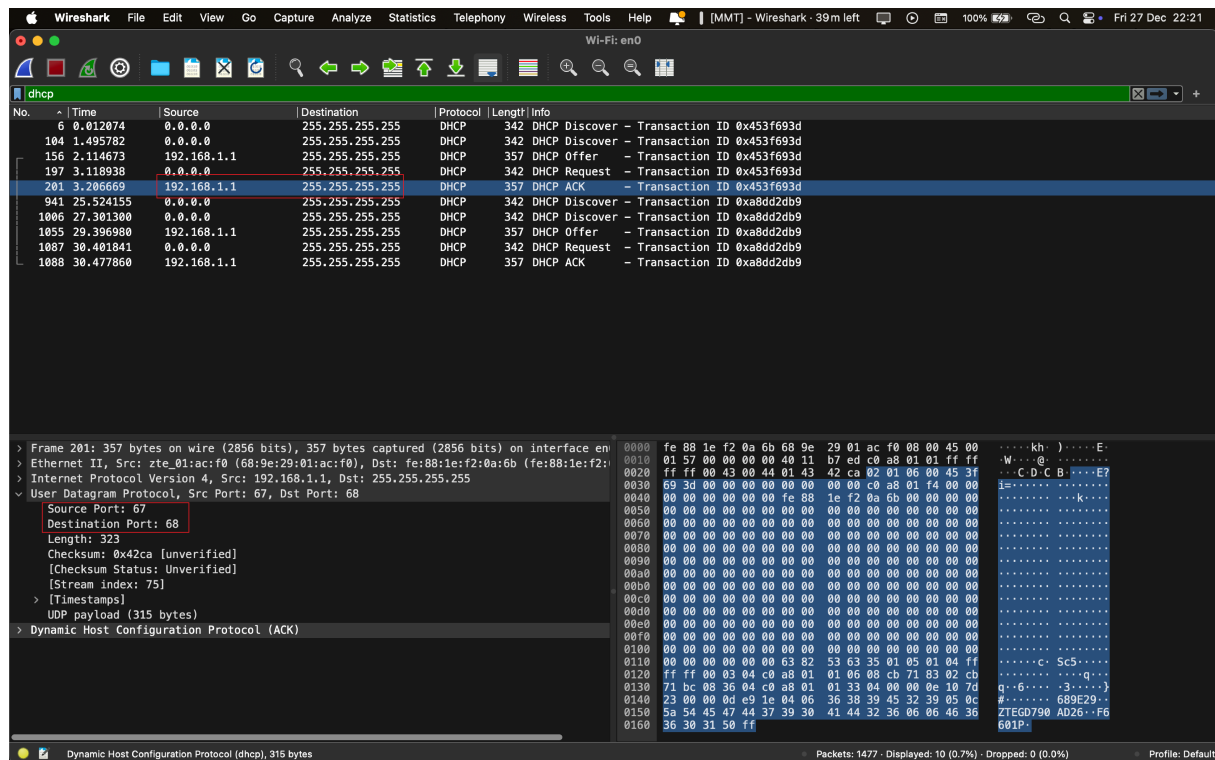
Hình 20: Chi tiết gói tin DHCP Discover



Hình 21: Chi tiết gói tin DHCP Offer



Hình 22: Chi tiết gói tin DHCP Request



Hình 23: Chi tiết gói tin DHCP ACK

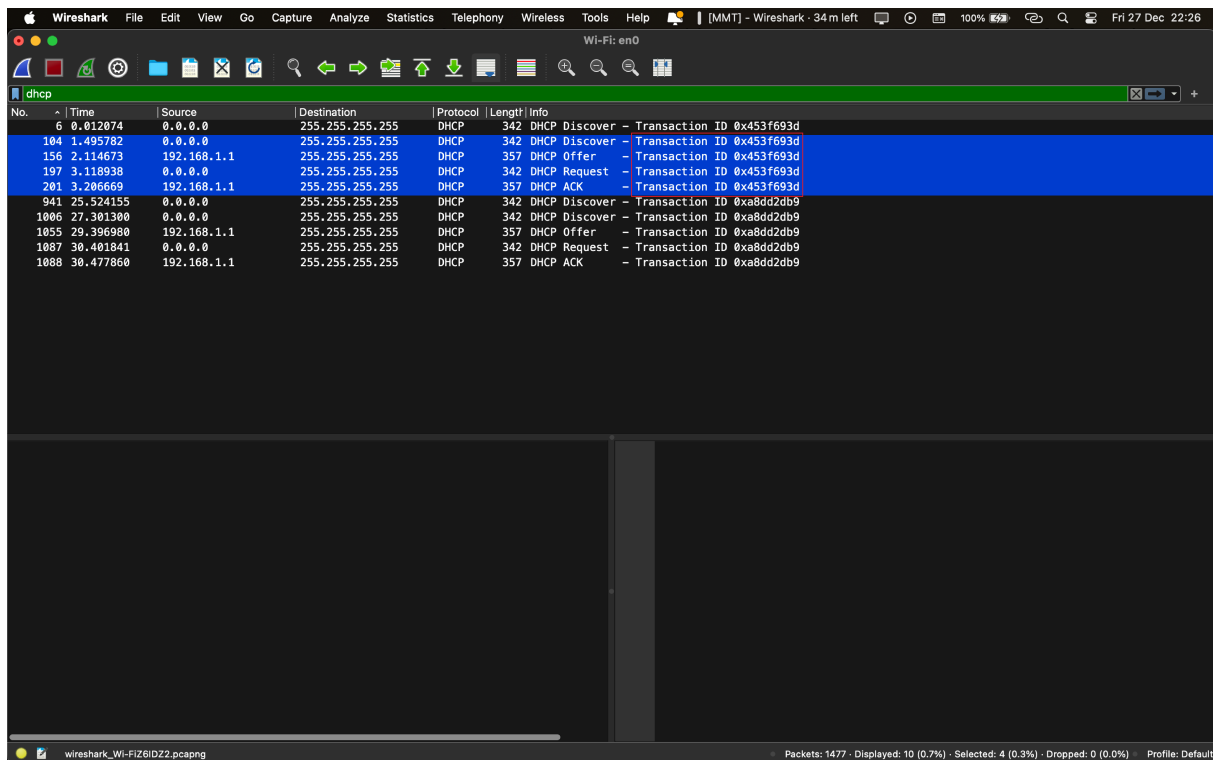
3.5 Chỉ ra điểm khác biệt giữa các giá trị trong gói tin DHCP Discover và DHCP Request

Tiêu chí	DHCP Discover	DHCP Request
Mục đích	Gửi bởi Client để tìm kiếm DHCP Server có sẵn	Gửi bởi Client để yêu cầu xác nhận IP được cấp phát từ DHCP Offer
Địa chỉ IP	Không có địa chỉ IP	Chứa địa chỉ IP yêu cầu (chỉ định trong DHCP Offer)
Thông tin trong gói tin	Địa chỉ MAC của máy khách	Địa chỉ IP được yêu cầu
Quá trình giao tiếp	Gửi khi bắt đầu yêu cầu IP	Gửi sau khi nhận DHCP Offer
Trạng thái máy chủ	Máy khách chưa nhận IP	Máy khách đã nhận ít nhất một Offer

Bảng 4: So sánh giữa DHCP Discover và DHCP Request[1]

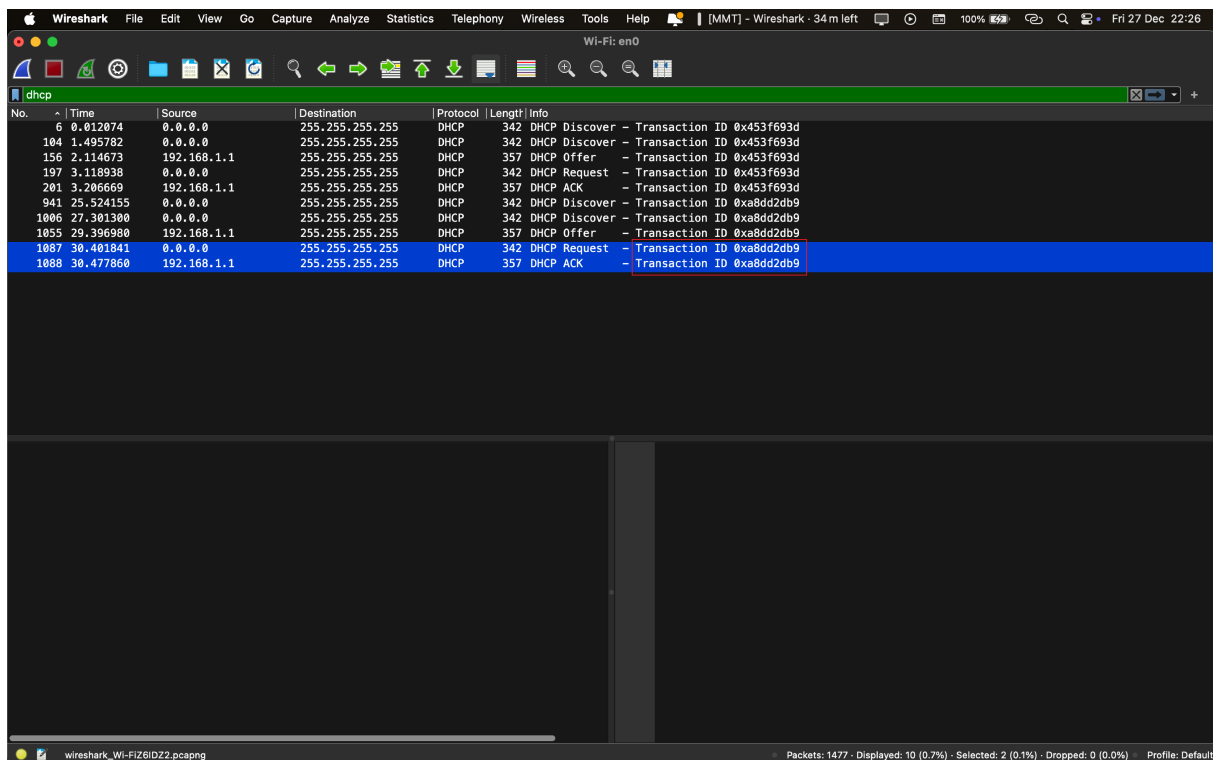
3.6 Cho biết giá trị Transaction-ID của 4 gói tin (Discover/Offer/Request/ACK) trong đợt xin cấp IP đầu tiên, giá trị Transaction-ID của 2 gói tin (Request/ACK) trong đợt xin cấp IP lần hai. Mục đích sử dụng của transaction-ID là gì?

Đợt xin cấp IP đầu tiên, giá trị Transaction-ID của 4 gói tin (Discover/Offer/Request/ACK): **0x453f693d**



Hình 24: Giá trị Transaction-ID của 4 gói tin lần xin cấp IP thứ nhất

Đợt xin cấp IP thứ hai, giá trị Transaction-ID của 2 gói tin (Request/ACK): **0xa8dd2db9**

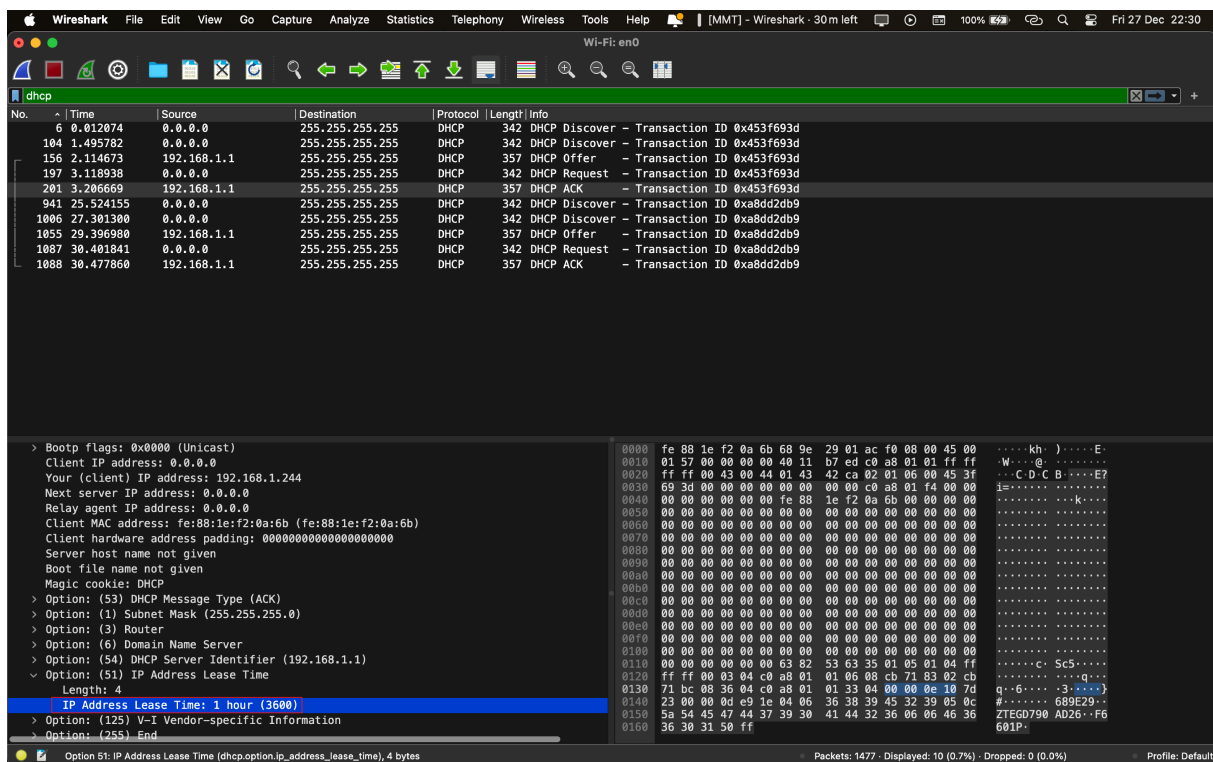


Hình 25: Giá trị Transaction-ID của 2 gói tin lần xin cấp IP thứ hai

Mục đích sử dụng của transaction-ID: Transaction-ID được DHCP client tạo ra ngẫu nhiên phân biệt các phiên giao tiếp (transactions) giữa DHCP client và DHCP server. Nó đảm bảo rằng mỗi yêu cầu và phản hồi DHCP là duy nhất và không bị nhầm lẫn với các phiên khác[2].

3.7 Cho biết ý nghĩa trường thông tin lease-time, giá trị của trường này trong gói tin bắt được là bao nhiêu?

- **Ý nghĩa trường thông tin lease-time:** Lease Time (thời gian thuê) là một giá trị được DHCP server cung cấp cho DHCP client, biểu thị khoảng thời gian mà client được phép sử dụng địa chỉ IP được cấp. Sau khi thời gian này kết thúc, client phải gia hạn (renew) địa chỉ IP hoặc trả lại địa chỉ đó để server có thể cấp phát cho thiết bị khác. [3]
- Giá trị của trường này trong gói tin bắt được là **1 hour (3600)**.



Hình 26: Thông tin lease-time của gói tin bắt được

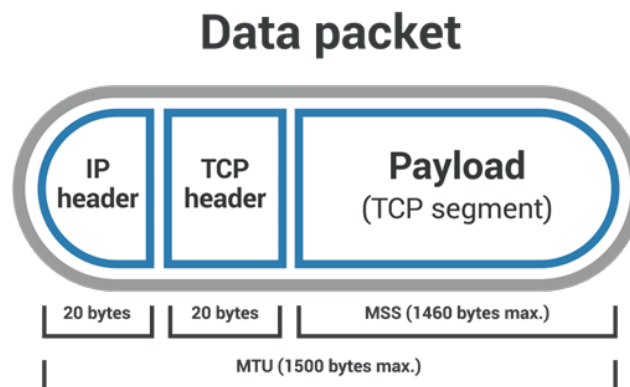
3.8 Trong gói tin DHCP bắt được, hãy chỉ ra trường giá trị cho biết có thông tin DHCP relay agent hay không? Nếu có hãy cho biết IP của DHCP relay agent.

- Trong gói tin DHCP bắt được, trường giá trị cho biết có thông tin DHCP relay agent hay không là Relay agent IP address.
- Địa chỉ IP của trường này có giá trị là 0.0.0.0, nghĩa là DHCP server và DHCP client thuộc cùng một mạng cục bộ (LAN) và DHCP server trực tiếp giao tiếp với client mà không cần qua bất kỳ

thiết bị trung gian nào để chuyển tiếp gói tin nên **không có Relay Agent** tham gia vào quá trình gói tin DHCP này.

4 ICMP

4.1 MSS (Maximum Segment Size) là gì? Liên quan gì đến MTU (Maximum Transmission Unit)?



Hình 27: Data packet headers and payload - TCP segment and MSS

MSS (Maximum Segment Size): Là kích thước tối đa của dữ liệu (**payload**) mà một gói **TCP** có thể chứa. **MSS** được giới hạn để dữ liệu không vượt quá khả năng xử lý của mạng.

MTU (Maximum Transmission Unit): Là kích thước lớn nhất của gói dữ liệu (bao gồm cả header) mà mạng có thể truyền qua.

Mối quan hệ giữa MSS và MTU:

MSS phụ thuộc trực tiếp vào **MTU** và được tính như sau:

$$\text{MSS} = \text{MTU} - (\text{IP header size} + \text{TCP header size})$$

Kích thước header thường gặp:

- **IP Header:** 20 bytes.
- **TCP Header:** 20 bytes.

Ví dụ:

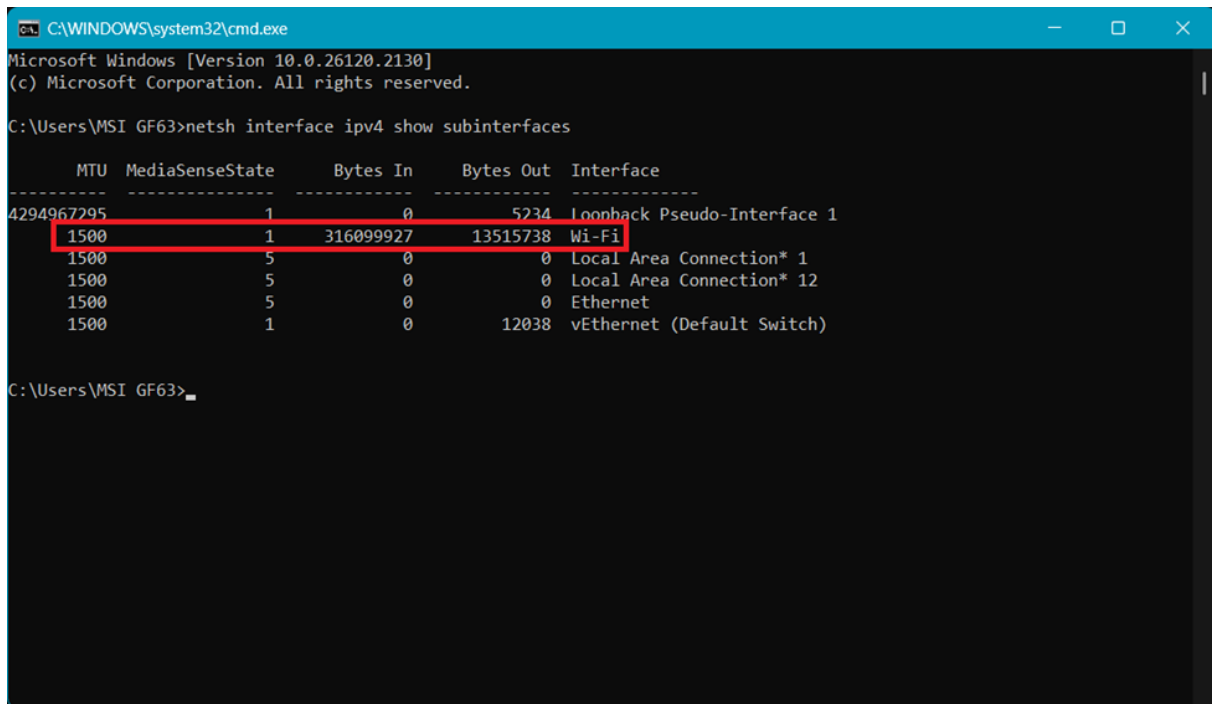
Với $\text{MTU} = 1500$:

$$\text{MSS} = 1500 - (20 + 20) = 1460 \text{ bytes.}$$

Nếu sử dụng MTU khác: Ví dụ $MTU = 1400$:

$$MSS = 1400 - (20 + 20) = 1360 \text{ bytes.}$$

4.2 Cho biết MTU trên card mạng kết nối internet của máy bạn là bao nhiêu (có chụp hình)?



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26120.2130]
(c) Microsoft Corporation. All rights reserved.

C:\Users\MSI GF63>netsh interface ipv4 show subinterfaces

    MTU  MediaSenseState      Bytes In      Bytes Out  Interface
-----
4294967295      1              0             5234  Loopback Pseudo-Interface 1
1500            1  316099927    13515738  Wi-Fi
1500            5              0             0  Local Area Connection* 1
1500            5              0             0  Local Area Connection* 12
1500            5              0             0  Ethernet
1500            1              0            12038  vEthernet (Default Switch)

C:\Users\MSI GF63>
```

Hình 28: MTU của card mạng

MTU trên card mạng kết nối internet (Wifi) của máy có giá trị là **1500 bytes**.

Cách kiểm tra MTU:

Để xem thông tin này, cần mở **Command Prompt** và gõ lệnh:

```
netsh interface ipv4 show subinterfaces
```

4.3 Kích thước MTU là 1500, cho biết kích thước của các phần của gói ICMP tại tầng mạng cơ bản (network) là bao nhiêu? (cho biết cách tính toán của bạn để có kết quả của từng phần)

Với kích thước $MTU = 1500$ bytes, ta sẽ tính kích thước các phần của gói ICMP tại tầng Network, bao gồm IPv4 Header, ICMP Header và ICMP Payload.

4.3.1 Kích thước của IPv4 Header

IPv4 Header có kích thước cố định là **20 bytes**. Đây là phần mở đầu trong tầng Network, chứa thông tin định tuyến như địa chỉ IP nguồn và đích.

4.3.2 Kích thước của ICMP Header

ICMP Header có kích thước cố định là **8 bytes**. Bao gồm các trường:

- **Type:** 1 byte
- **Code:** 1 byte
- **Checksum:** 2 bytes
- **Identifier:** 2 bytes
- **Sequence number:** 2 bytes

4.3.3 Kích thước của ICMP Data

Tổng kích thước tầng Network phải nằm trong giới hạn MTU:

$$\text{MTU} = \text{IPv4 Header} + \text{ICMP Header} + \text{ICMP Payload}$$

$$1500 = 20 + 8 + \text{ICMP Payload}$$

$$\text{ICMP Payload} = 1500 - 20 - 8 = 1472 \text{ bytes.}$$

Tuy nhiên, đó là kích thước tối đa của **ICMP Payload**. Thông thường, kích thước này chỉ là **32 bytes** để tránh lãng phí và hạn chế phân mảnh. Do đó, kích thước thông thường của các phần tại tầng Network là:

$$20 + 8 + 32 = 60 \text{ bytes.}$$

4.3.4 Kết luận

Kích thước tại tầng Network có thể là:

- **1500 bytes (tối đa)**
- **60 bytes (thông thường)**

4.4 Kích thước MTU là 1500, cho biết kích thước của các phần của gói ICMP tại tầng Data Link cơ bản là bao nhiêu? (cho biết cách tính toán của bạn để có kết quả của từng phần)

Tại tầng **Data Link**, dữ liệu từ tầng **Network** được đóng gói thành khung Ethernet với các thành phần sau:

- **Ethernet Header:** Gồm địa chỉ **MAC nguồn** (6 bytes), **MAC đích** (6 bytes), và trường **EtherType** (2 bytes). Tổng là 14 bytes.
- **Dữ liệu (Payload):** Dữ liệu chính từ tầng **Network**. Với **MTU = 1500**, payload là dữ liệu tầng Network: 1500 bytes. Hoặc 60 bytes theo kích thước thông thường.
- **Ethernet Trailer:** Gồm **Frame Check Sequence (FCS)** dùng để kiểm tra lỗi. Có kích thước cố định là 4 bytes.

Tổng kích thước của khung Ethernet tại tầng Data Link:

$$\text{Ethernet Header} + \text{Payload} + \text{Ethernet Trailer} = 14 + 1500 + 4 = 1518 \text{ bytes.}$$

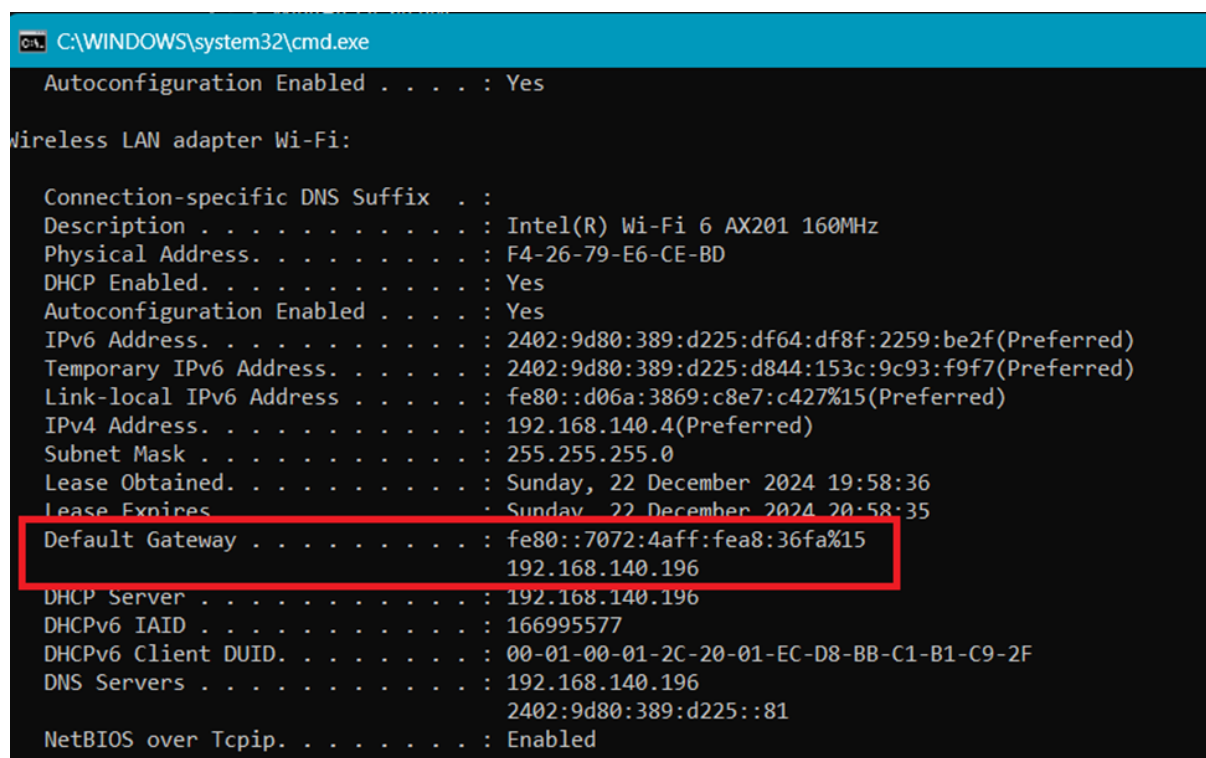
Hoặc:

$$14 + 60 + 4 = 78 \text{ bytes.}$$

Ngoài ra, trong mạng có hỗ trợ **VLAN (802.1Q tagging)**, phần header sẽ thêm 4 bytes để chứa thông tin VLAN, làm kích thước khung Ethernet tăng lên:

$$1522 \text{ bytes (hoặc } 82 \text{ bytes).}$$

Kiểm tra địa chỉ IP của Default Gateway



```
C:\WINDOWS\system32\cmd.exe
Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : 
Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : F4-26-79-E6-CE-BD
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : 2402:9d80:389:d225:df64:df8f:2259:be2f(Preferred)
Temporary IPv6 Address. . . . . : 2402:9d80:389:d225:d844:153c:9c93:f9f7(Preferred)
Link-local IPv6 Address . . . . . : fe80::d06a:3869:c8e7:c427%15(Preferred)
IPv4 Address. . . . . : 192.168.140.4(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Sunday, 22 December 2024 19:58:36
Lease Expires . . . . . : Sunday, 22 December 2024 20:58:35
Default Gateway . . . . . : fe80::7072:4aff:fea8:36fa%15
                          192.168.140.196
DHCP Server . . . . . : 192.168.140.196
DHCPv6 IAID . . . . . : 166995577
DHCPv6 Client DUID. . . . . : 00-01-00-01-2C-20-01-EC-D8-BB-C1-B1-C9-2F
DNS Servers . . . . . : 192.168.140.196
                          2402:9d80:389:d225::81
NetBIOS over Tcpip. . . . . : Enabled
```

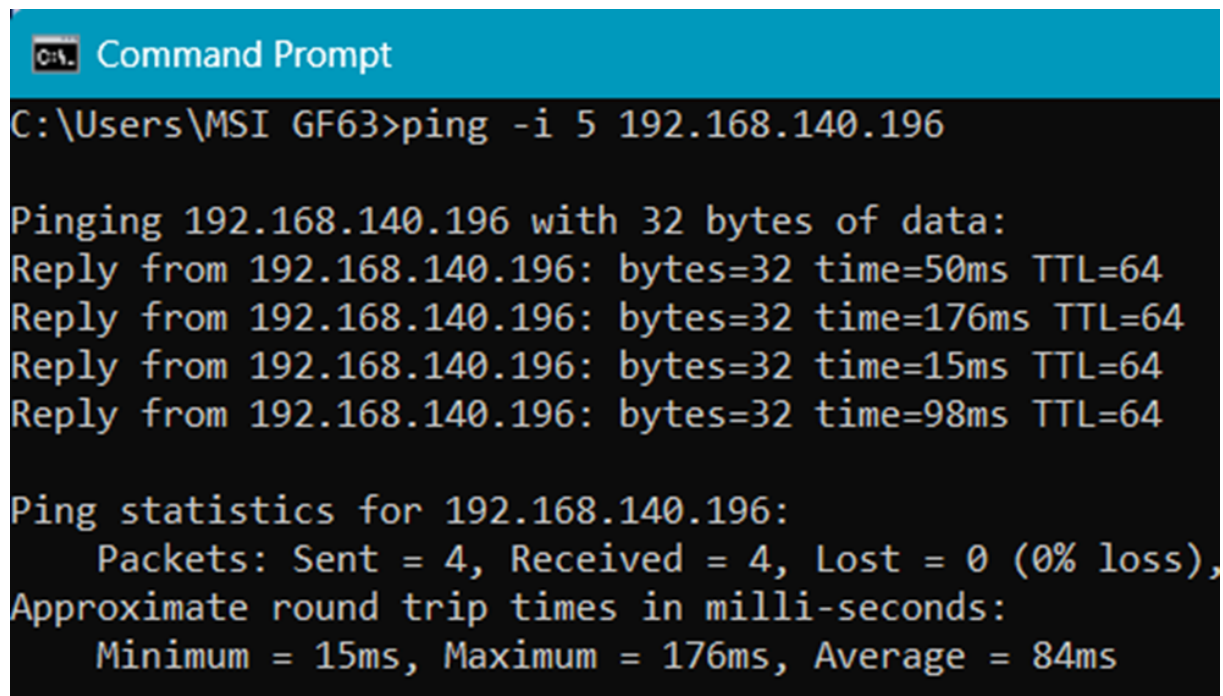
Hình 29: Default Gateway

Sau khi kiểm tra, **Default Gateway** của em là 192.168.140.196.

4.5 Câu lệnh ping -i 5 <ip của default gateway> và ping -l 33 <ip của default gateway>

4.5.1 Lệnh ping -i 5 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu? TTL (Time to live) là bao nhiêu?

Khi nhập lệnh ping -i 5 192.168.140.196, ta thu được kết quả sau:



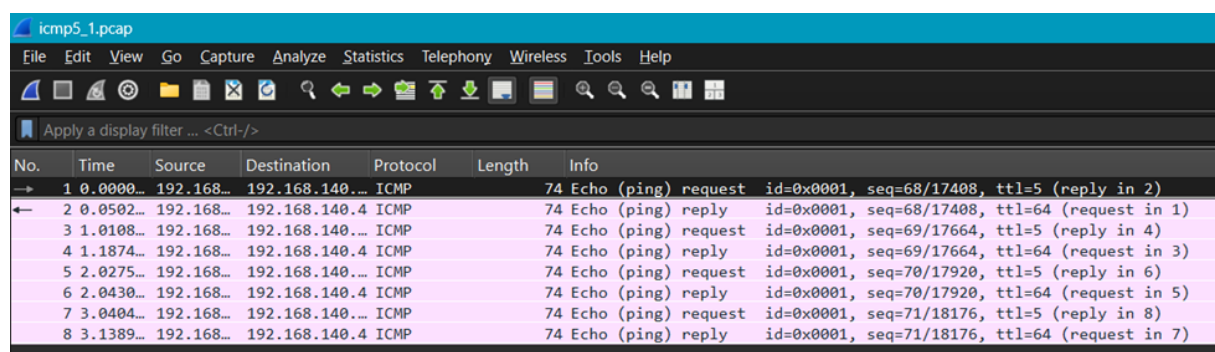
```

C:\Users\MSI GF63>ping -i 5 192.168.140.196

Pinging 192.168.140.196 with 32 bytes of data:
Reply from 192.168.140.196: bytes=32 time=50ms TTL=64
Reply from 192.168.140.196: bytes=32 time=176ms TTL=64
Reply from 192.168.140.196: bytes=32 time=15ms TTL=64
Reply from 192.168.140.196: bytes=32 time=98ms TTL=64

Ping statistics for 192.168.140.196:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 15ms, Maximum = 176ms, Average = 84ms
  
```

Hình 30: Câu lệnh ping -i 5 192.168.140.196



No.	Time	Source	Destination	Protocol	Length	Info
→ 1	0.0000...	192.168...	192.168.140...	ICMP	74	Echo (ping) request id=0x0001, seq=68/17408, ttl=5 (reply in 2)
← 2	0.0502...	192.168...	192.168.140.4	ICMP	74	Echo (ping) reply id=0x0001, seq=68/17408, ttl=64 (request in 1)
3	1.0108...	192.168...	192.168.140...	ICMP	74	Echo (ping) request id=0x0001, seq=69/17664, ttl=5 (reply in 4)
4	1.1874...	192.168...	192.168.140.4	ICMP	74	Echo (ping) reply id=0x0001, seq=69/17664, ttl=64 (request in 3)
5	2.0275...	192.168...	192.168.140...	ICMP	74	Echo (ping) request id=0x0001, seq=70/17920, ttl=5 (reply in 6)
6	2.0430...	192.168...	192.168.140.4	ICMP	74	Echo (ping) reply id=0x0001, seq=70/17920, ttl=64 (request in 5)
7	3.0404...	192.168...	192.168.140...	ICMP	74	Echo (ping) request id=0x0001, seq=71/18176, ttl=5 (reply in 8)
8	3.1389...	192.168...	192.168.140.4	ICMP	74	Echo (ping) reply id=0x0001, seq=71/18176, ttl=64 (request in 7)

Hình 31: File icmp5_1.pcap

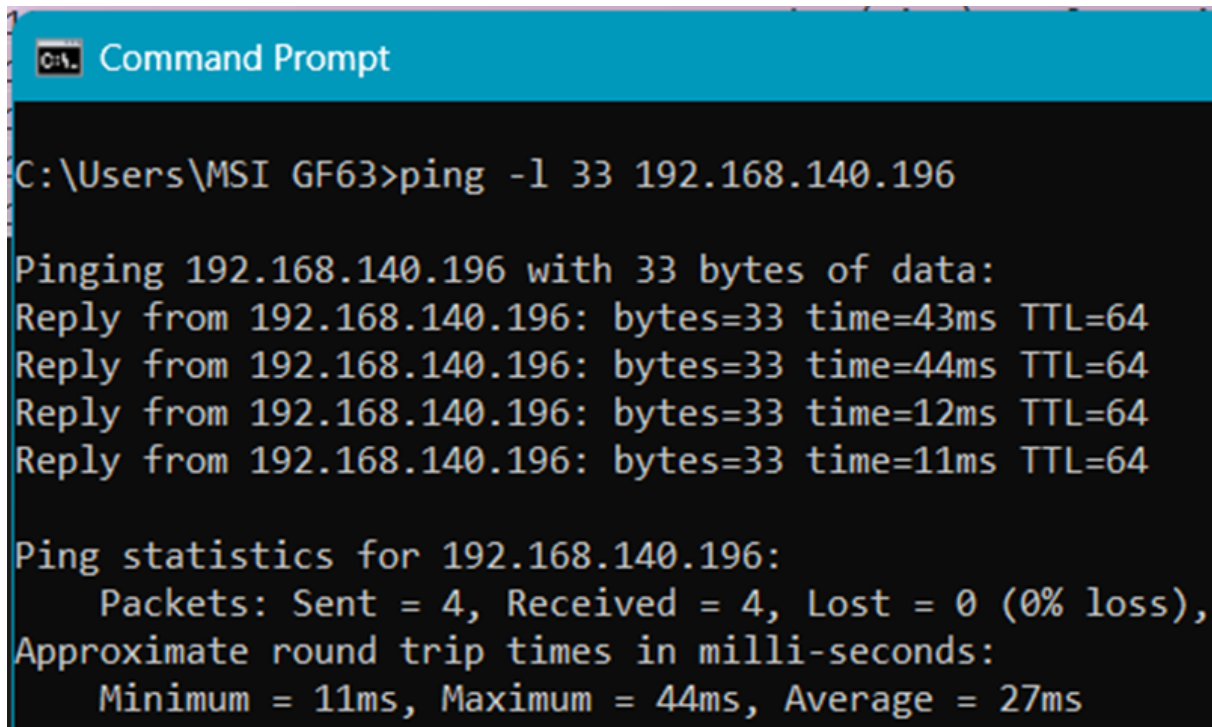
Khi này, **ICMP payload** bằng 32 bytes, **Time to live (TTL)** của gói tin **request** là 5, và của gói tin **reply** là 64.

Giải thích: Vì có lệnh -i, lệnh này trên Windows dùng để điều chỉnh **Time to live (TTL)**, còn trên

Linux hoặc macOS, lệnh này dùng để điều chỉnh thời gian gửi giữa các gói tin.

4.5.2 Lệnh ping -l 33 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu? TTL (Time to live) là bao nhiêu?

Khi nhập lệnh ping -l 33 192.168.140.196, ta được kết quả sau:

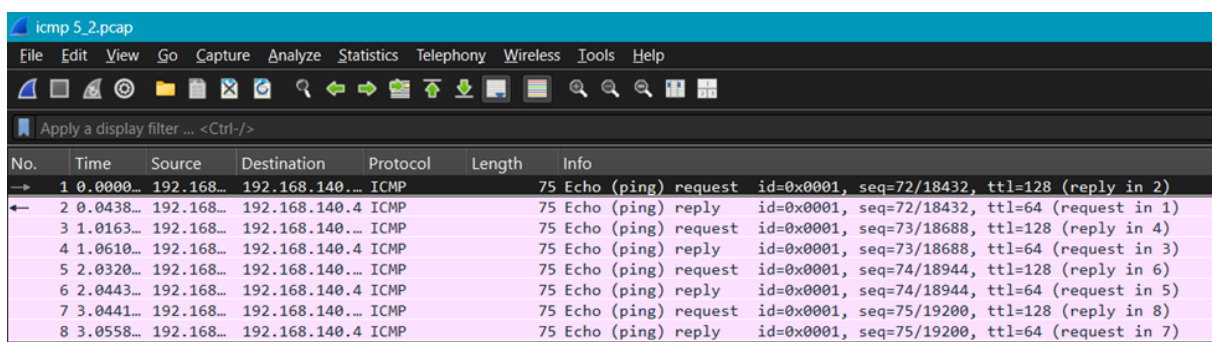


```
C:\Users\MSI GF63>ping -l 33 192.168.140.196

Pinging 192.168.140.196 with 33 bytes of data:
Reply from 192.168.140.196: bytes=33 time=43ms TTL=64
Reply from 192.168.140.196: bytes=33 time=44ms TTL=64
Reply from 192.168.140.196: bytes=33 time=12ms TTL=64
Reply from 192.168.140.196: bytes=33 time=11ms TTL=64

Ping statistics for 192.168.140.196:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 11ms, Maximum = 44ms, Average = 27ms
```

Hình 32: Câu lệnh ping -l 33 192.168.140.196



No.	Time	Source	Destination	Protocol	Length	Info
→ 1	0.0000...	192.168...	192.168.140...	ICMP	75	Echo (ping) request id=0x0001, seq=72/18432, ttl=128 (reply in 2)
← 2	0.0438...	192.168...	192.168.140.4	ICMP	75	Echo (ping) reply id=0x0001, seq=72/18432, ttl=64 (request in 1)
3	1.0163...	192.168...	192.168.140...	ICMP	75	Echo (ping) request id=0x0001, seq=73/18688, ttl=128 (reply in 4)
4	1.0610...	192.168...	192.168.140.4	ICMP	75	Echo (ping) reply id=0x0001, seq=73/18688, ttl=64 (request in 3)
5	2.0320...	192.168...	192.168.140...	ICMP	75	Echo (ping) request id=0x0001, seq=74/18944, ttl=128 (reply in 6)
6	2.0443...	192.168...	192.168.140.4	ICMP	75	Echo (ping) reply id=0x0001, seq=74/18944, ttl=64 (request in 5)
7	3.0441...	192.168...	192.168.140...	ICMP	75	Echo (ping) request id=0x0001, seq=75/19200, ttl=128 (reply in 8)
8	3.0558...	192.168...	192.168.140.4	ICMP	75	Echo (ping) reply id=0x0001, seq=75/19200, ttl=64 (request in 7)

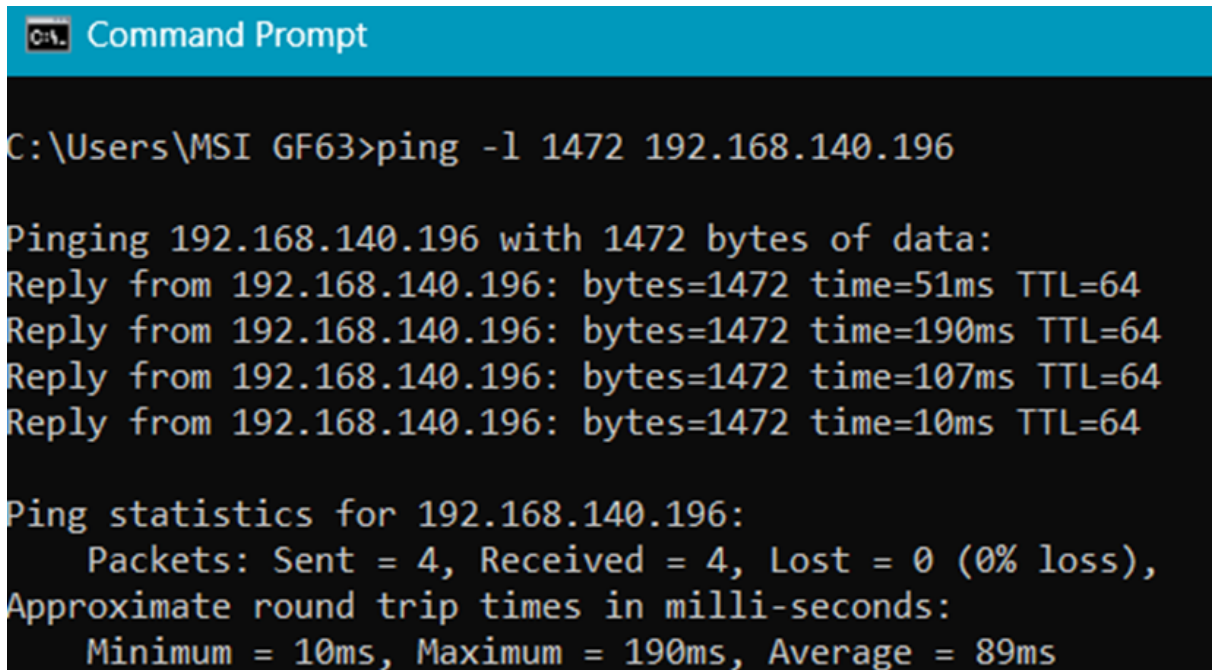
Hình 33: File icmp5_2.pcap

Khi này, **ICMP payload** bằng 33 bytes, **Time to live (TTL)** của gói tin request là 128, và của gói tin reply là 64.

Giải thích: Vì có lệnh -l, lệnh này trên Windows dùng để điều chỉnh kích thước của **ICMP payload**, còn trên Linux hoặc macOS, lệnh này dùng -s để điều chỉnh kích thước.

4.6 Lệnh ping -l 1472 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu?

Khi nhập lệnh ping -l 1472 192.168.140.196, ta được kết quả như sau:



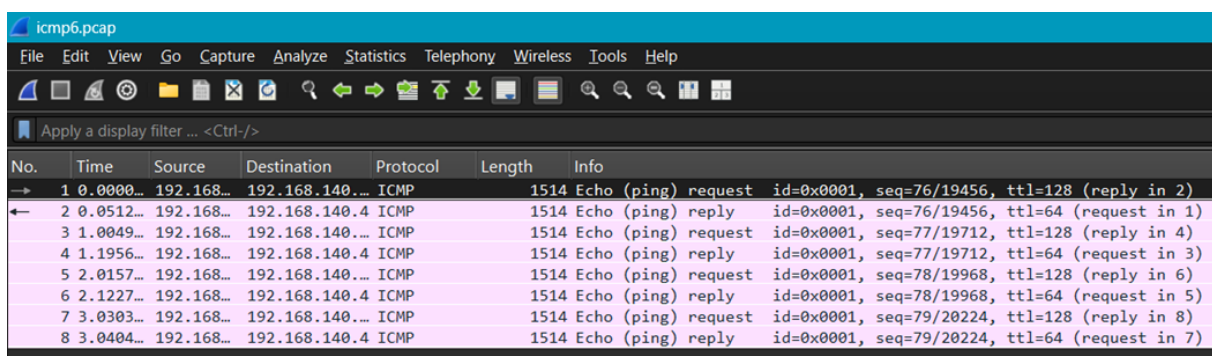
```

C:\Users\MSI GF63>ping -l 1472 192.168.140.196

Pinging 192.168.140.196 with 1472 bytes of data:
Reply from 192.168.140.196: bytes=1472 time=51ms TTL=64
Reply from 192.168.140.196: bytes=1472 time=190ms TTL=64
Reply from 192.168.140.196: bytes=1472 time=107ms TTL=64
Reply from 192.168.140.196: bytes=1472 time=10ms TTL=64

Ping statistics for 192.168.140.196:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 10ms, Maximum = 190ms, Average = 89ms
  
```

Hình 34: Lệnh ping -l 1472 192.168.140.196



No.	Time	Source	Destination	Protocol	Length	Info
→ 1	0.0000...	192.168...	192.168.140...	ICMP	1514	Echo (ping) request id=0x0001, seq=76/19456, ttl=128 (reply in 2)
← 2	0.0512...	192.168...	192.168.140.4	ICMP	1514	Echo (ping) reply id=0x0001, seq=76/19456, ttl=64 (request in 1)
3	1.0049...	192.168...	192.168.140...	ICMP	1514	Echo (ping) request id=0x0001, seq=77/19712, ttl=128 (reply in 4)
4	1.1956...	192.168...	192.168.140.4	ICMP	1514	Echo (ping) reply id=0x0001, seq=77/19712, ttl=64 (request in 3)
5	2.0157...	192.168...	192.168.140...	ICMP	1514	Echo (ping) request id=0x0001, seq=78/19968, ttl=128 (reply in 6)
6	2.1227...	192.168...	192.168.140.4	ICMP	1514	Echo (ping) reply id=0x0001, seq=78/19968, ttl=64 (request in 5)
7	3.0303...	192.168...	192.168.140...	ICMP	1514	Echo (ping) request id=0x0001, seq=79/20224, ttl=128 (reply in 8)
8	3.0404...	192.168...	192.168.140.4	ICMP	1514	Echo (ping) reply id=0x0001, seq=79/20224, ttl=64 (request in 7)

Hình 35: File icmp6.pcap

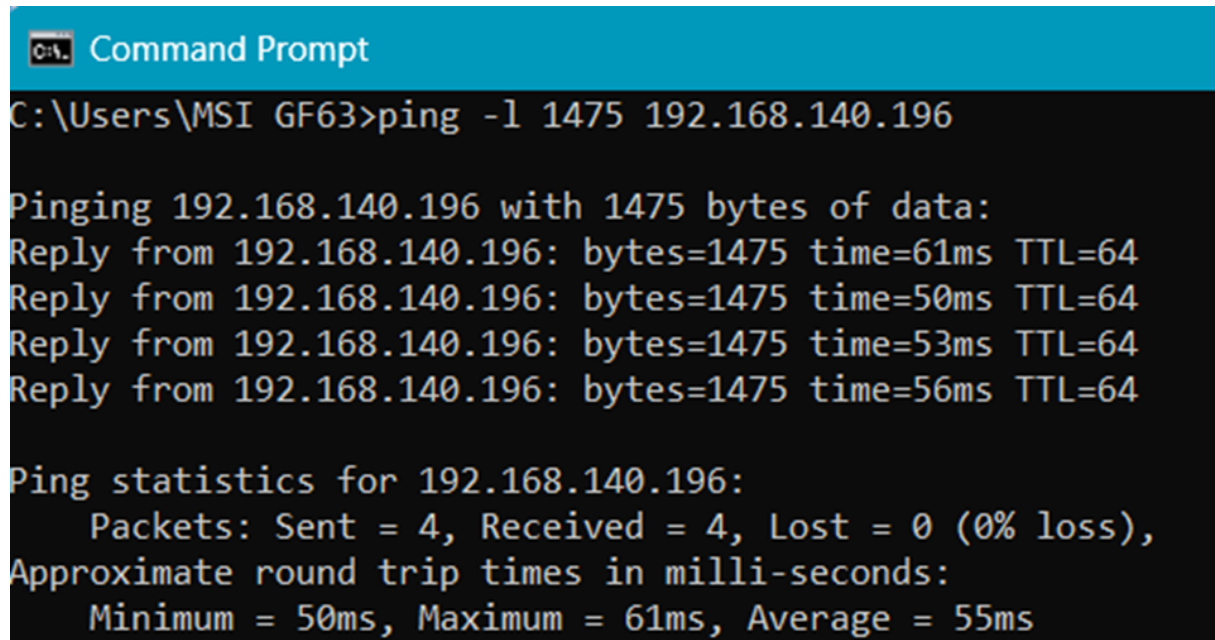
Khi này, ICMP payload bằng 1472 bytes.

Giải thích: Kích thước gói tin tại tầng Network lúc này bằng IP header + ICMP header + ICMP payload = 20 + 8 + 1472 = 1500.

Vừa bằng với MTU nên gói tin được truyền đi mà không bị phân mảnh.

4.7 Lệnh ping -l 1475 192.168.140.196. Cho biết lượng data trong ICMP (ICMP payload) là bao nhiêu? Gói tin khác gì so với câu 6 bạn đã bắt ở trên và giải thích sự khác nhau đó?

Khi nhập lệnh ping -l 1475 192.168.140.196, ta được kết quả như sau:



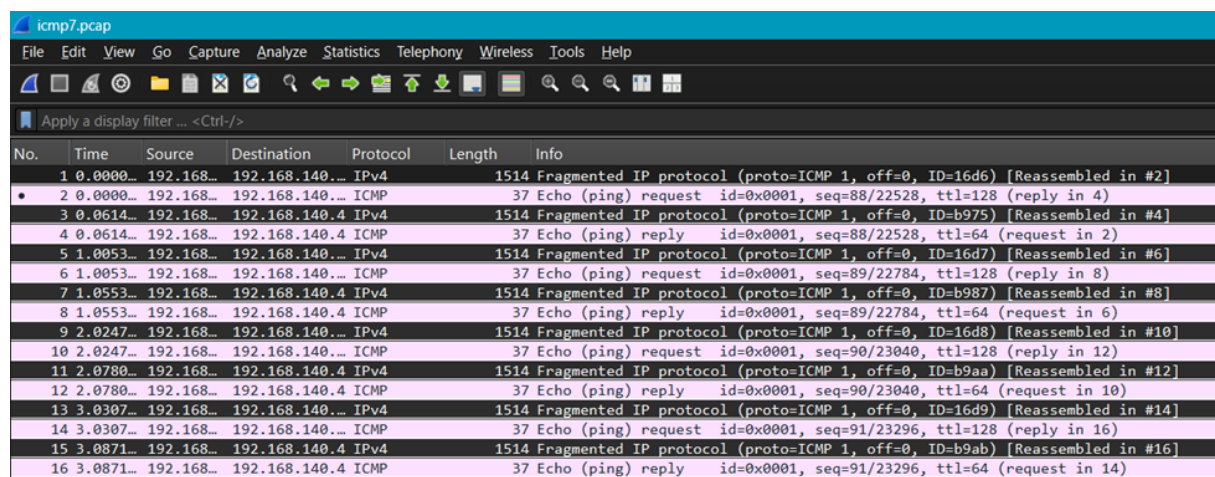
```

C:\Users\MSI GF63>ping -l 1475 192.168.140.196

Pinging 192.168.140.196 with 1475 bytes of data:
Reply from 192.168.140.196: bytes=1475 time=61ms TTL=64
Reply from 192.168.140.196: bytes=1475 time=50ms TTL=64
Reply from 192.168.140.196: bytes=1475 time=53ms TTL=64
Reply from 192.168.140.196: bytes=1475 time=56ms TTL=64

Ping statistics for 192.168.140.196:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 50ms, Maximum = 61ms, Average = 55ms
  
```

Hình 36: Lệnh ping -l 1475 192.168.140.196



No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000...	192.168...	192.168.140...	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=16d6) [Reassembled in #2]
2	0.0000...	192.168...	192.168.140...	ICMP	37	Echo (ping) request id=0x0001, seq=88/22528, ttl=128 (reply in 4)
3	0.0614...	192.168...	192.168.140.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=b975) [Reassembled in #4]
4	0.0614...	192.168...	192.168.140.4	ICMP	37	Echo (ping) reply id=0x0001, seq=88/22528, ttl=64 (request in 2)
5	1.0053...	192.168...	192.168.140...	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=16d7) [Reassembled in #6]
6	1.0053...	192.168...	192.168.140...	ICMP	37	Echo (ping) request id=0x0001, seq=89/22784, ttl=128 (reply in 8)
7	1.0553...	192.168...	192.168.140.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=b987) [Reassembled in #8]
8	1.0553...	192.168...	192.168.140.4	ICMP	37	Echo (ping) reply id=0x0001, seq=89/22784, ttl=64 (request in 6)
9	2.0247...	192.168...	192.168.140...	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=16d8) [Reassembled in #10]
10	2.0247...	192.168...	192.168.140...	ICMP	37	Echo (ping) request id=0x0001, seq=90/23040, ttl=128 (reply in 12)
11	2.0780...	192.168...	192.168.140.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=b9aa) [Reassembled in #12]
12	2.0780...	192.168...	192.168.140.4	ICMP	37	Echo (ping) reply id=0x0001, seq=90/23040, ttl=64 (request in 10)
13	3.0307...	192.168...	192.168.140...	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=16d9) [Reassembled in #14]
14	3.0307...	192.168...	192.168.140...	ICMP	37	Echo (ping) request id=0x0001, seq=91/23296, ttl=128 (reply in 16)
15	3.0871...	192.168...	192.168.140.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=b9ab) [Reassembled in #16]
16	3.0871...	192.168...	192.168.140.4	ICMP	37	Echo (ping) reply id=0x0001, seq=91/23296, ttl=64 (request in 14)

Hình 37: File icmp7.pcap

Khi này, **ICMP payload** hiển thị là 1475 bytes nhưng thực chất chỉ còn 3 bytes, do 1472 bytes trước đó đã bị phân mảnh vào một gói tin với Protocol là **IPv4** đã được gửi trước. Khi nhận lại reply với 2 gói **IPv4** và **ICMP** với kích thước tương tự như 2 gói đã gửi.

Lệnh ping -l 1475 192.168.140.196 gửi gói **ICMP Echo Request** đến 192.168.140.196 với kích thước dữ liệu (**ICMP payload**) là 1475 bytes.

- Tùy chọn `-l` chỉ định kích thước của **ICMP payload**, tức là 1475 bytes dữ liệu thực tế mà gói tin mang theo.
- Lượng dữ liệu trong **ICMP (ICMP payload)** là 1475 bytes.

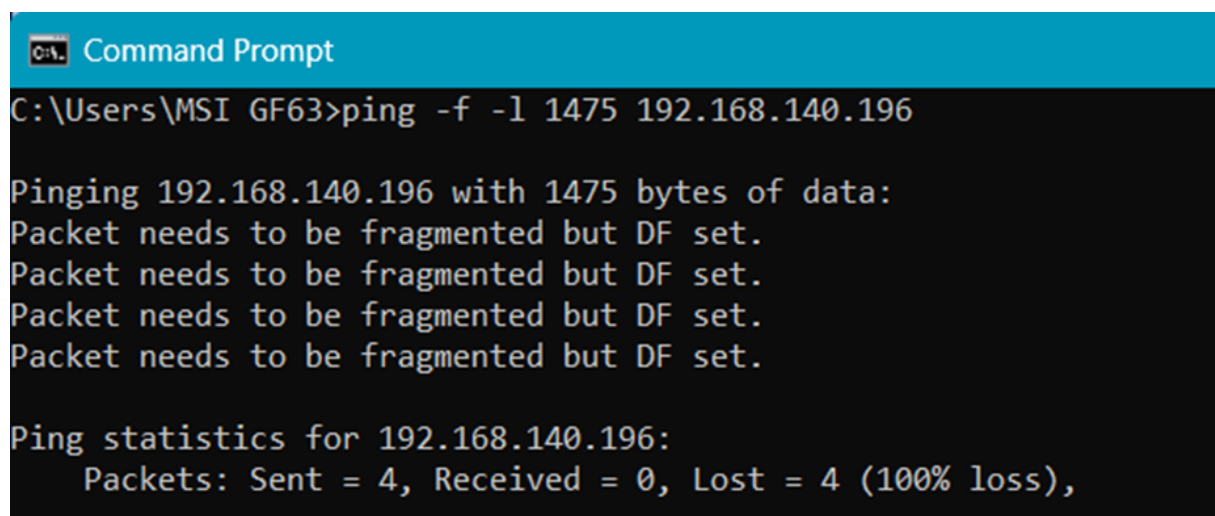
Giải thích sự khác nhau giữa 2 lệnh ping:

- Khi thực hiện lệnh `ping -l 1472 192.168.140.196`, gói tin **ICMP** có kích thước **payload** 1472 bytes, tổng kích thước là 1500 bytes (bao gồm 8 bytes cho **ICMP header** và 20 bytes cho **IP header**), đúng với **MTU** của mạng Ethernet, nên không bị phân mảnh.
- Ngược lại, với lệnh `ping -l 1475 192.168.140.196`, gói tin có **payload** 1475 bytes, tổng kích thước là 1503 bytes (bao gồm 8 bytes cho **ICMP header** và 20 bytes cho **IP header**), vượt quá **MTU** và sẽ bị phân mảnh để truyền qua mạng.

Vậy, sự khác biệt giữa `icmp6.pcap` (1472 bytes) và `icmp7.pcap` (1475 bytes) là gói tin `icmp6.pcap` không bị phân mảnh, còn `icmp7.pcap` bị phân mảnh.

4.8 Lệnh `ping -f -l 1475 192.168.1.8`. Sau khi bắt gói tin bạn thấy vấn đề gì và giải thích kết quả mà bạn có được?

Khi nhập lệnh `ping -f -l 1475 192.168.140.196`, ta được kết quả như sau:



```
Command Prompt
C:\Users\MSI GF63>ping -f -l 1475 192.168.140.196

Pinging 192.168.140.196 with 1475 bytes of data:
Packet needs to be fragmented but DF set.
Packet needs to be fragmented but DF set.
Packet needs to be fragmented but DF set.
Packet needs to be fragmented but DF set.

Ping statistics for 192.168.140.196:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Hình 38: Lệnh `ping -f -l 1475 192.168.140.196`

Khi này đã có thông báo gói tin cần được phân mảnh do vượt quá **MTU** là 1500. Nhưng chúng ta lại có lệnh “-f” nghĩa là không được phân mảnh gói tin nên không thể vượt qua tầng **Network** và mất gói tin ngay tại thiết bị gửi do gói tin vượt quá **MTU**. Lệnh `ping -f -l 1475 192.168.1.8` gửi gói **ICMP** với kích thước dữ liệu (**ICMP payload**) là 1475 bytes. Tùy chọn `-f` yêu cầu gói tin không được phép phân mảnh, và `-l 1475` chỉ định kích thước của **ICMP payload** là 1475 bytes.

- Kết quả nhận được khi thực hiện lệnh trên là:

- "Packet needs to be fragmented but DF set": Thông báo này cho biết gói tin **ICMP** yêu cầu phân mảnh nhưng cờ **DF** (Don't Fragment) đã được thiết lập, nên gói tin không thể phân mảnh và bị từ chối.
- Kết quả thống kê cho thấy 4 gói tin đã được gửi đi nhưng không có gói tin nào nhận lại, tất cả đều bị mất (**100% loss**), xác nhận rằng gói tin không thể truyền qua mạng.
- Giải thích: Lỗi xảy ra vì gói tin có kích thước 1503 bytes vượt quá giới hạn **MTU** (1500 bytes), và cờ **-f** (**Don't Fragment**) được thiết lập, yêu cầu gói tin không được phép phân mảnh. Do đó, gói tin không thể truyền qua mạng và bị từ chối.

Tài liệu

- [1] wikipedia. *Dynamic Host Configuration Protocol*. https://en.wikipedia.org/wiki/Dynamic_Host_Configuration_Protocol. Lý thuyết gói tin DHCP. Truy cập 26-12-2024.
- [2] quora. *What is the function of a transaction ID in a DHCP packet?* <https://www.quora.com/What-is-the-function-of-a-transaction-ID-in-a-DHCP-packet>. Tham khảo cách xem thông tin gói tin DHCP bằng phần mềm Wireshark. Truy cập 26-12-2024.
- [3] thietbimanggiare.com. *Giao thức DHCP là gì? Vai trò và cách thức hoạt động của DHCP Server*. <https://thietbimanggiare.com/dynamic-host-configuration-protocol-dhcp/>. Tham khảo lý thuyết trường Lease-time và trường Replay Agent. Truy cập 26-12-2024.
- [4] wireshark. *DHCP*. <https://wiki.wireshark.org/DHCP>. Hướng dẫn xem gói tin DHCP. Truy cập 26-12-2024.
- [5] thietbimanggiare.com. *Giao thức DHCP là gì? Vai trò và cách thức hoạt động của DHCP Server*. <https://thietbimanggiare.com/dynamic-host-configuration-protocol-dhcp/>. Tham khảo lý thuyết trường Lease-time và trường Replay Agent. Truy cập 26-12-2024.
- [6] cloudflare. *What is the Internet Control Message Protocol (ICMP)?* <https://www.cloudflare.com/learning/ddos/glossary/internet-control-message-protocol-icmp/>. Tham khảo giao thức ICMP và các phần liên quan. Truy cập 22-12-2024.
- [7] University of Massachusetts. *Wireshark HTTP Lab (Version 8.0)*. https://www-net.cs.umass.edu/wireshark-labs/Wireshark_HTTP_v8.0.pdf. Tham khảo cách xem thông tin gói tin HTTP bằng phần mềm Wireshark. Truy cập 27-12-2024.
- [8] geeksforgeeks. *DNS in Wireshark*. <https://www.geeksforgeeks.org/dns-in-wireshark/>. Tham khảo cách xem thông tin gói tin DNS bằng phần mềm Wireshark. Truy cập 27-12-2024.